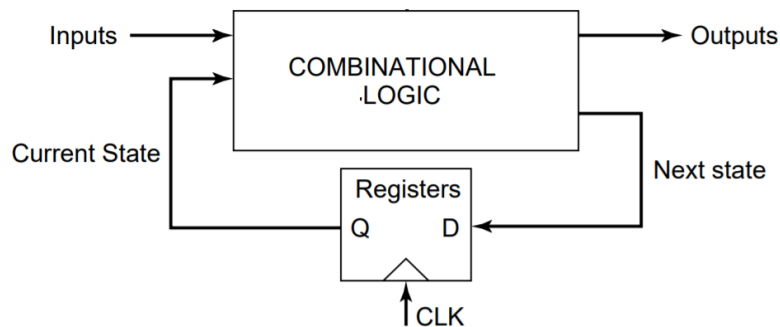


Sequential Logic Circuits

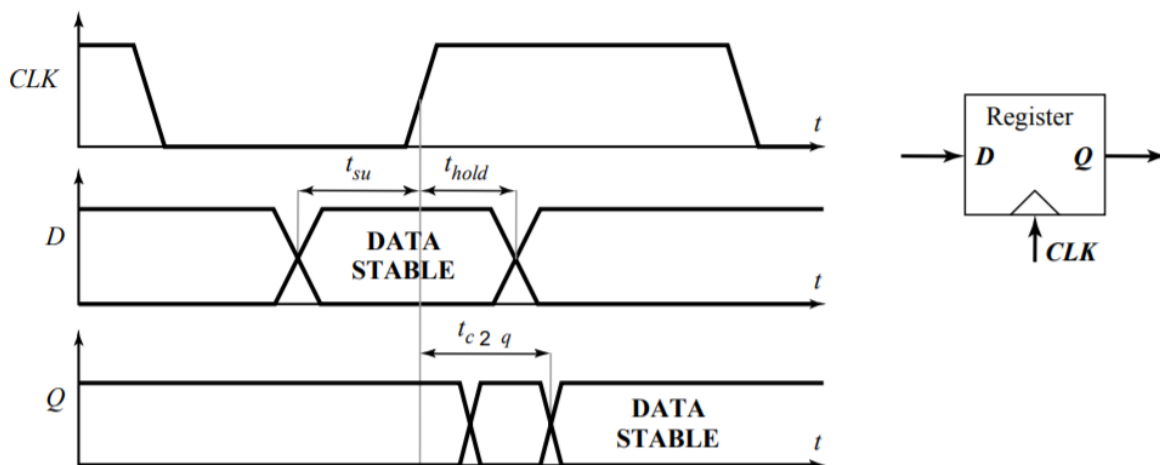
Logica sequenziale

Sono logiche nelle quali le uscite non dipendono istantaneamente dagli ingressi (logiche combinatorie) ma hanno degli "stati"



un circuito sequenziale è composto da degli ingressi che vengono elaborati mediante una logica combinatoria. L'esito di questa elaborazione porta a delle uscite + lo stato nel quale dovrà trovarsi la logica al prossimo ciclo.

Definizione dei tempi



t_{su} → tempo minimo per il quale il dato deve rimanere stabile prima del fronte di clock in modo che venga campionato correttamente

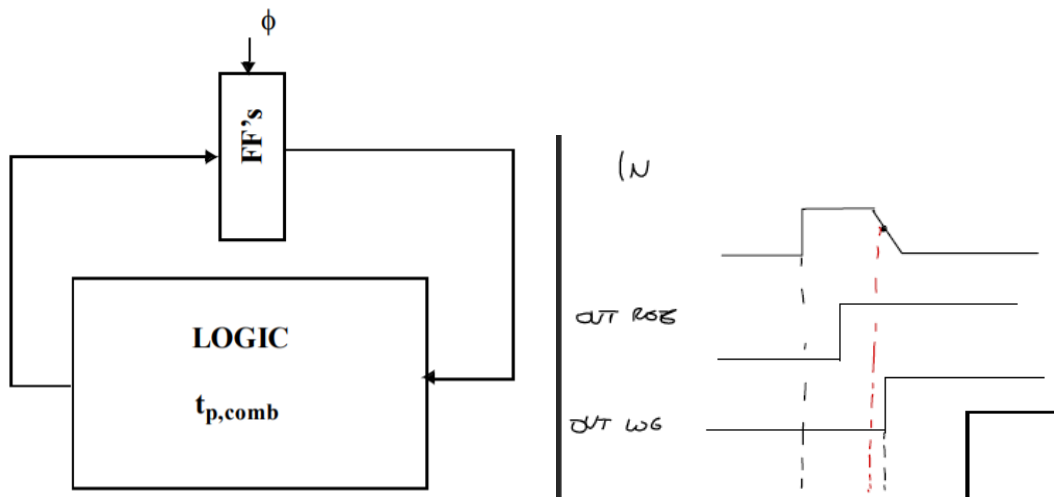
t_{hold} → tempo minimo per il quale il dato deve rimanere stabile dopo del fronte di clock in modo che venga campionato correttamente

t_{c-q} (t di clock to q) → tempo di caso peggiore nel quale il dato viene copiato in uscita (tempo dopo il quale l'uscita è stabile) *(a seguito del fronte di clock)*

Questi tempi impongono un vincolo sulla frequenza di clock che dovrà avere un periodo:

$$T_{\text{clock}} > t_{\text{su}} + t_{\text{c-q}} + t_{\text{prop-logica}} \quad (\text{tempo di propagazione della logica combinatoria})$$

inoltre è presente un tempo definito t_{cd} (contamination delay) che definisce il tempo dopo cui l'uscita varia a fronte della variazione di ingresso



Per far sì che l'ingresso arrivi così com'è al registro (entrando dallo stesso) bisognerà avere che:

$$t_{\text{cdreg}} + t_{\text{cdlogic}} > t_{\text{hold}}$$

Se quel tempo fosse minore del tempo di hold, avrei delle perturbazioni sul nodo di ingresso prima che finisca il tempo di hold e questo non va bene proprio per la definizione di tempo di hold.

Tipologie di registri

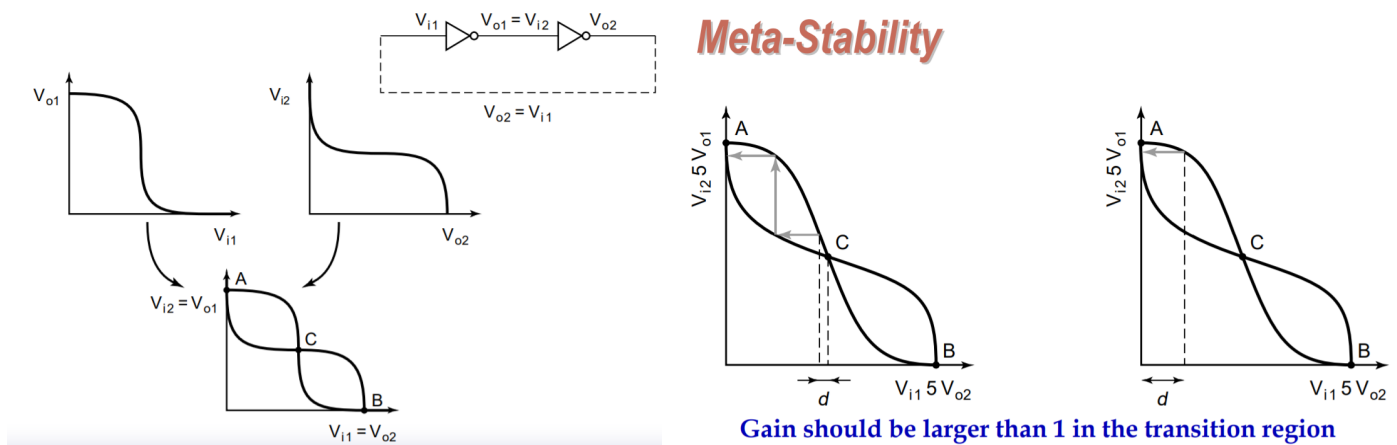
- **latch**
 - è **level sensitive**
 - positive
 - trasparente (fa passare il dato) → $CLK = 1$
 - hold (mantiene il dato in memoria) → $CLK = 0$
 - negative
 - trasparente → $CLK = 0$
 - hold → $CLK = 1$
- **registro**
 - è **edge-triggered** ovvero è trasparente al fronte di salita (positive edge) al fronte di discesa (negative edge)

I registri possono essere:

- **statici** → preservano lo stato finché c'è alimentazione
- **dinamici** → dati immagazzinati per un breve periodo di tempo (storage nei condensatori e per il leakage vanno refreshati come ad esempio la DRAM)

Bistabile

è la serie retroazionata di un numero pari di inverter (ovvero l'uscita dell'ultimo è l'ingresso del primo).

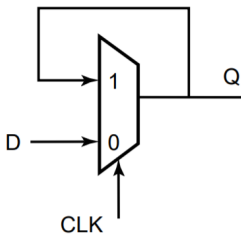


Abbiamo graficato la caratteristica del primo inverter, la caratteristica del secondo non è altro che quella del primo ribaltata in quanto l'uscita di 1 è l'ingresso di 2 e viceversa. Unendo le caratteristiche trovo 3 punti di intersezione che rappresentano i punti di lavoro del circuito.

- **A e B sono punti stabili** → la caratteristica dell'inverter deve essere >1 in modo che abbia effetto la proprietà rigenerativa dei segnali digitali (ovvero che se un segnale analogico non è riconoscibile né come 1 né come 0, ponendo in cascata più inverter il segnale si sposta verso il relativo valore digitale)
- **C è il punto metastabile**
 - se si introduce del rumore in ingresso al primo inverter provocando uno spostamento del punto di lavoro nella caratteristica, si vede che il punto di lavoro si sposta verso uno dei due punti stabili A o B

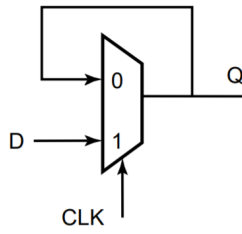
Mux-Based Latch

Negative latch
(transparent when CLK= 0)

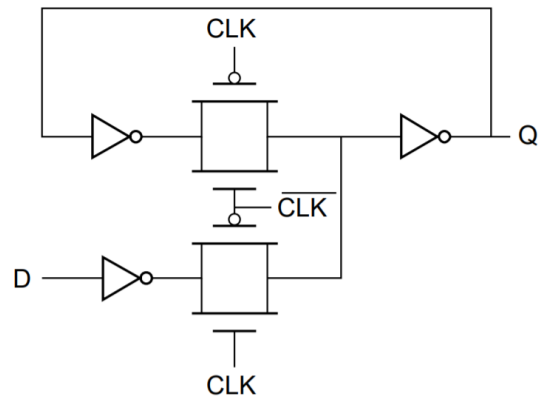


$$Q = Clk \cdot Q + \overline{Clk} \cdot In$$

Positive latch
(transparent when CLK= 1)

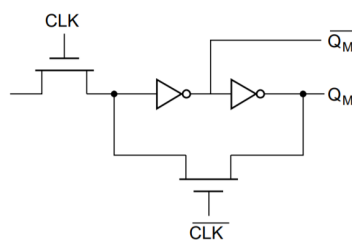


$$Q = \overline{Clk} \cdot Q + Clk \cdot In$$

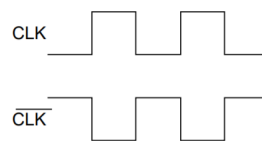


- l'inverter dopo D consente al canale di ingresso di essere direzionale (il pass-transistor che segue è per sua natura bidirezionale)
- in questo circuito il **clock-load** (il carico attaccato al clock come numero di mosfet) è alto e quindi la distribuzione del clock può avere dei ritardi con possibili **clock overlapping** (sovrapposizioni del clock)

Mux-Based Latch



NMOS only

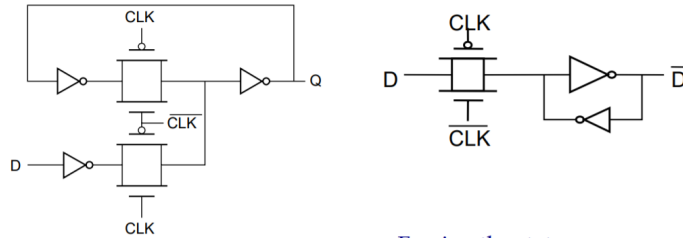


Non-overlapping clocks

- in questo caso il clock load è basso e c'è meno rischio di clock overlapping
- ho solo nMOS (no pass-transistor complementare) e quindi ho il problema del trasferimento dell' "1 debole" $V_{dd} - V_t$ (**signal integrity**) che porta al consumo statico degli invertitori che seguono (vedi pass-transistor)

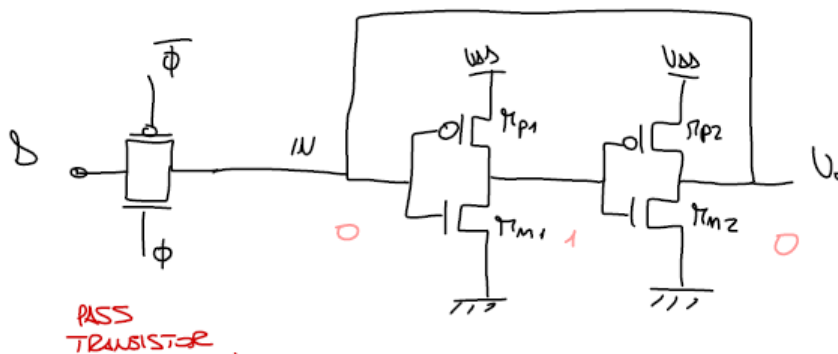
Writing into a Static Latch

Use the clock as a decoupling signal,
that distinguishes between the transparent and opaque states



Converting into a MUX

Forcing the state
(can implement as NMOS-only)



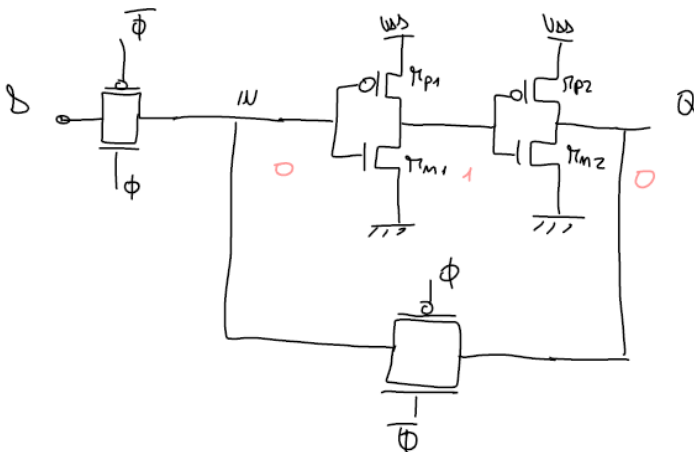
È un latch

per scrivere e imporre
è ingetto

In questo circuito ho un bistabile che viene pilotato da un pass-transistor complementare. Quando sul latch è immagazzinato per esempio uno zero e voglio imporre un uno in ingresso, dovrò fare in modo che il **rapporto di forza** tra il pass-transistor e il transistor M_{n2} sia più favorevole al pass-transistor (in quanto M_{n2} tenderà a tenere a massa il nodo di uscita).

Per evitare questa contesa si può progettare il circuito nel seguente modo, nel quale ho due pass-transistor:

- uno che gestisce la **scrittura** sul latch
- l'altro (che è in controfase con l'altro) che gestisce la **lettura**

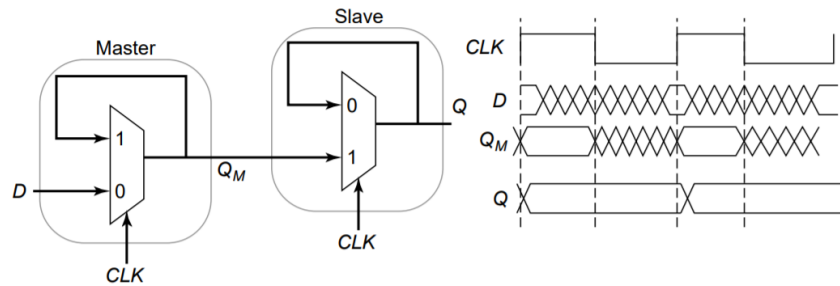


$\phi = 1 \Rightarrow$ scrivo l'ingetto e la retroazione non c'è
(TRANSPARENT: copia l'ingetto)

$\phi = 0 \Rightarrow$ la retroazione e forte (HOLD)

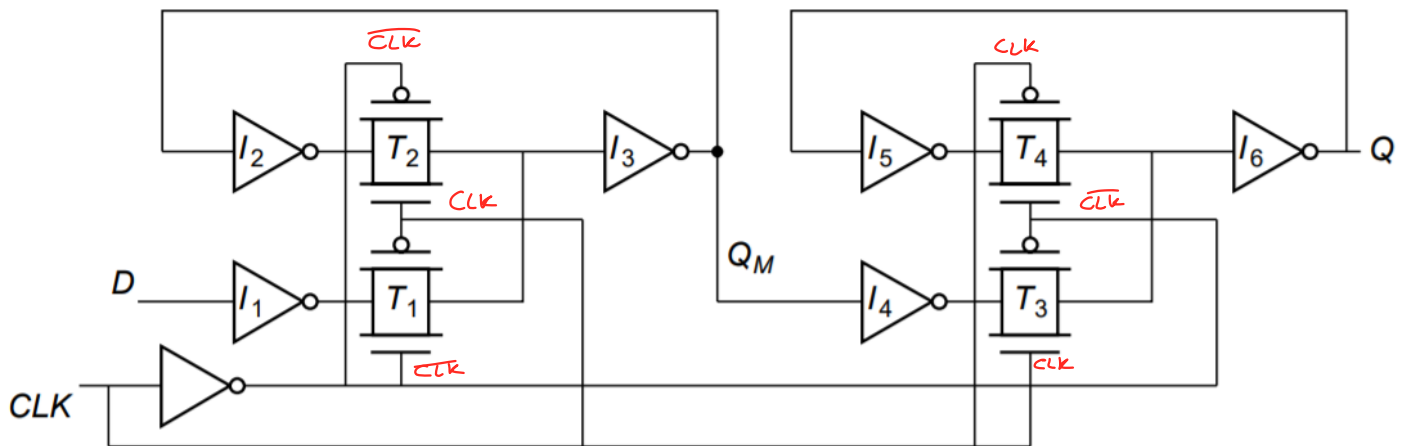
$$Q = \phi \cdot D + \bar{\phi} \cdot Q$$

Master-Slave (edge-triggered) Register



Two opposite latches trigger on edge
Also called master-slave latch pair

Multiplexer-based latch pair

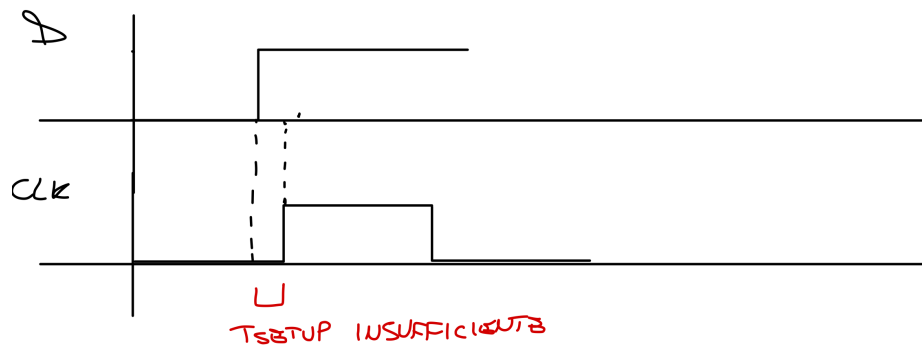


- i due multiplexer-based latch lavorano in controfase in modo che lo slave possa copiare l'ingresso campionato dal master
- T1 è un **pass-transistor di tipo transmission gate** ovvero un pass transistor complementare nei quali ai gate dei due mosfet c'è un segnale di controllo a sua volta complementare
 - quando il segnale di controllo che arriva al gate dell'nMOS è il clock allora si parla di **pass-transistor di tipo clock** (se fosse arrivato il clock negato sarebbe stato di tipo clock negato)
- la retroazione di I3 su I2 serve a mantenere stabile il bistabile
 - è necessario un t_{setup} in modo che D sia stabile mentre I2 sta valutando il suo ingresso
 - non c'è t_{hold} in quanto se $\text{CLK} = 1$, T1 è spento e il latch è in stato di hold
- I4 porta l'ingresso allo slave e rende unidirezionale il circuito → T3 non può pilotare a monte

FUNZIONAMENTO:

CLK = 0

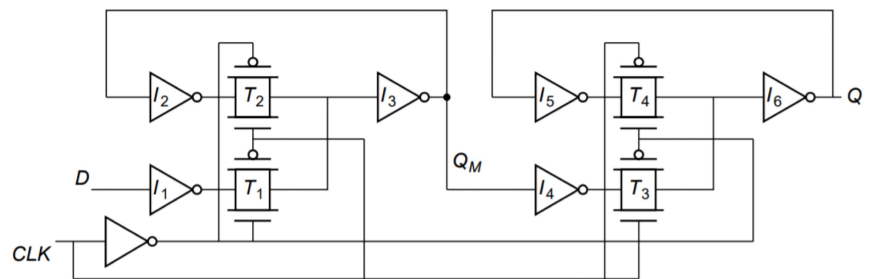
- T1 ON, T4 ON
- voglio che il dato arrivi fino all'uscita di I2 per essere immagazzinato nel bistabile
 - infatti quando avremo CLK = 1, T2 si accenderà ribadendo il dato a I3 (in quanto si vede che l'ingresso di I3 è logicamente uguale all'uscita di I2)
 - ci vuole tuttavia il tempo che il dato arrivi all'uscita di I2
 - supponiamo che D commuti da 0 a 1 poco prima che CLK = 1, se il dato arriva in Qm (che sarà uguale a uno) e in quel momento arriva il clock (perchè è molto ravvicinato al cambio del dato) I2 non è ancora riuscito a valutare l'uno e quindi valuterà lo zero (dato precedente) quando si accenderà T2 (ovvero Qm) al clk = 1



- questo porta a una contesa (chi vince?) in quanto
 - I2 valuta lo zero e impone un'uno in ingresso a I3
 - l'ingresso di I3 vale però zero (perchè il dato era uno)
 - la vincita della contesa dipende dalla variabilità dei transistor
- come azione cautelativa si impone che il dato rispetti il tempo di setup, ovvero che D rimanga stabile per un certo tempo prima dell'evento di clock, fino alla valutazione di I2 così che al livello di clk alto I2 ribadisca semplicemente il dato
- in questo caso il dato è arrivato sia in I2 che in I4 quindi il t_{c-q} è il tempo necessario per attraversare T3 e I6

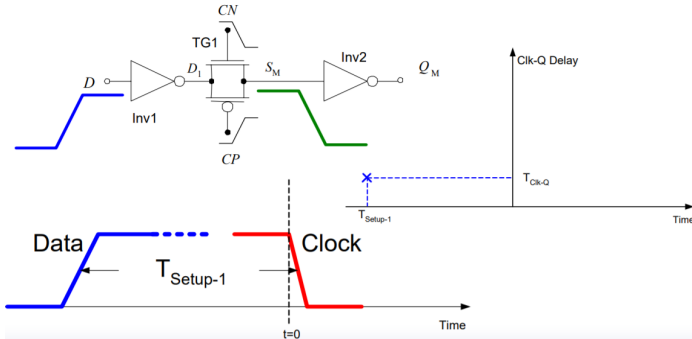
CLK = 1

- T2 ON, T3 ON
- il dato nel bistabile arriverà a T3 e poi all'uscita con I6



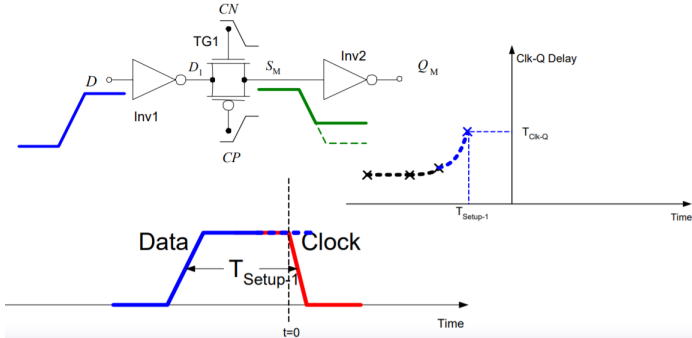
tempo di setup

Circuit before clock arrival (Setup-1 case)

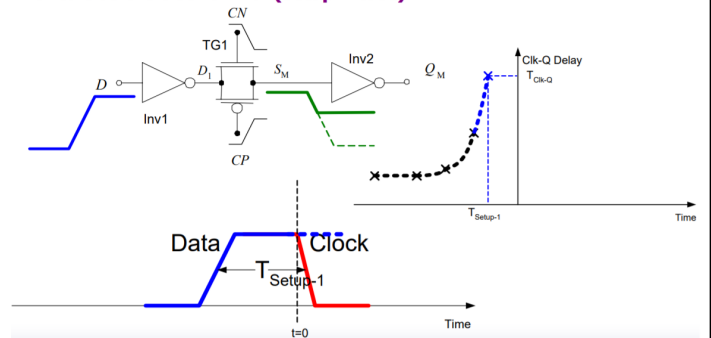


- D commuta “gradualmente” da 0 a 1
- inv 1 porterà in uscita una transizione 1 - 0
- al pass-transistor però arriva un segnale di clock che anch’esso non varia istantaneamente
- se il tempo di setup è troppo vicino alla variazione del clock, il pass-transistor non ha abbastanza tempo per far passare tutto il segnale in uscita da inv1
- aumenta quindi il t_{c-q} fino al fallimento

Circuit before clock arrival (Setup-1 case)

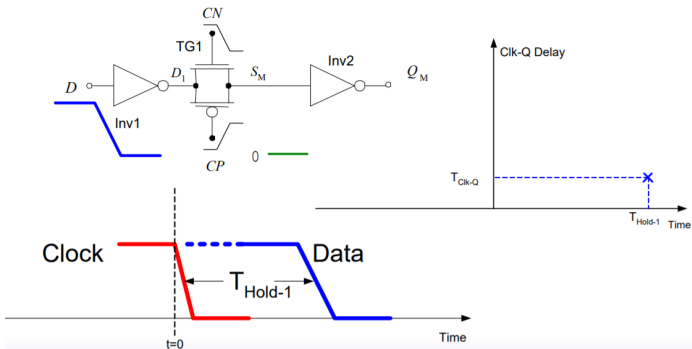


Circuit before clock arrival (Setup-1 case)



tempo di hold

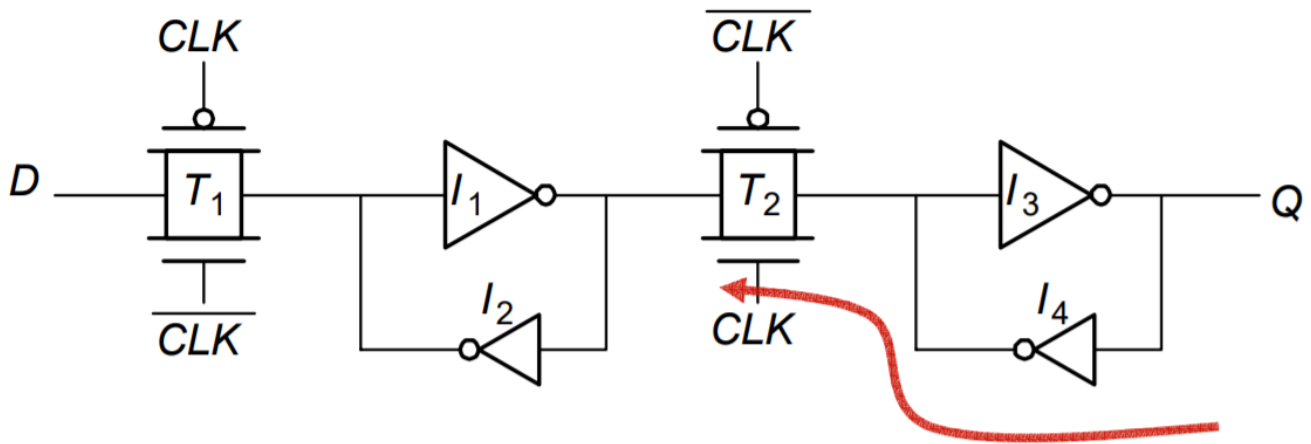
Hold-1 case



La variazione del tempo di hold permette di contrastare il problema della sovrapposizione delle fasi, pertanto se il dato rimane stabile dopo il campionamento per un certo tempo di hold, non ho il rischio che in caso di clock overlapping questo dato possa commutare nel frattempo.

Reduced Clock Load Master-Slave Register

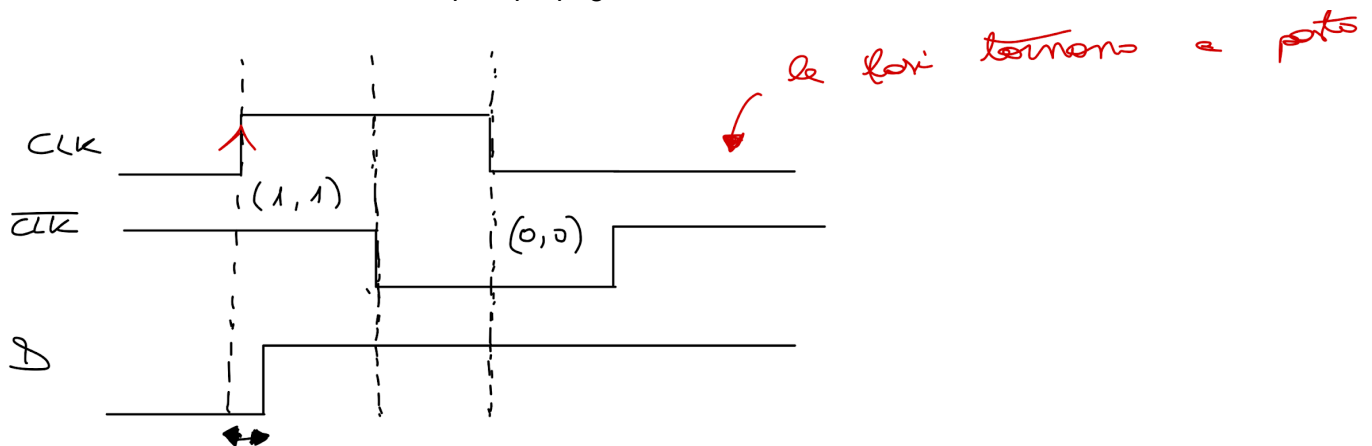
Nel circuito precedente il clock load era pesante (10 transistor attaccati al clock)



- I_2 (e I_4) progettato per essere debole ad esempio con dimensionamento minimo (evita la contesa) serve per assicurarsi che sia un bistabile
- è un registro positive edge-triggered statico
- non c'è più unidirezionalità (perdita di robustezza) → i pass-transistor devono prevalere sugli invertitori retorazionati
- problema della **reverse conduction**: lo slave può ripilotare il master

Clock overlapping

Quando il circuito legge due fasi che sono complementari può sorgere il problema che queste non commutino nello stesso istante a causa dei tempi di propagazione

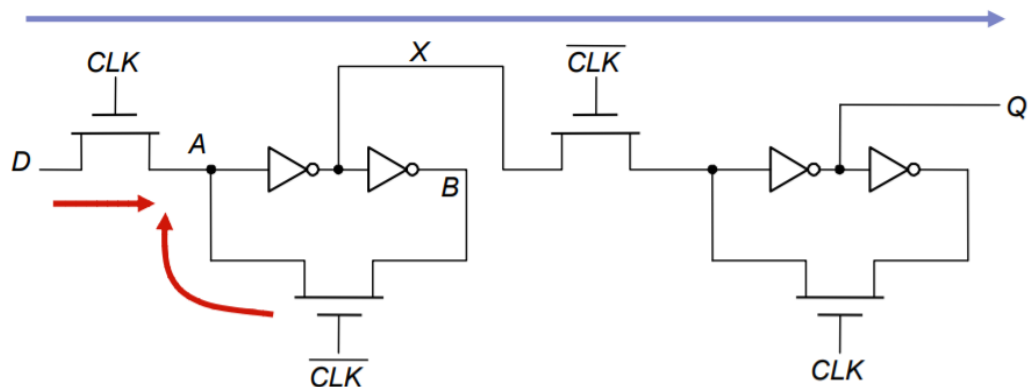


Quando ho delle sovrapposizioni:

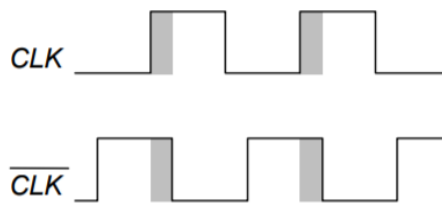
- $(1,1) \rightarrow$ tutti nMOS accesi
- $(0,0) \rightarrow$ tutti pMOS accesi

Se ho una sovrapposizione nell'istante di campionamento del master e D commuta a valle della transizione del clock, quando le fasi tornano a posto lo slave copia questo valore

Avoiding Clock Overlap



(a) Schematic diagram

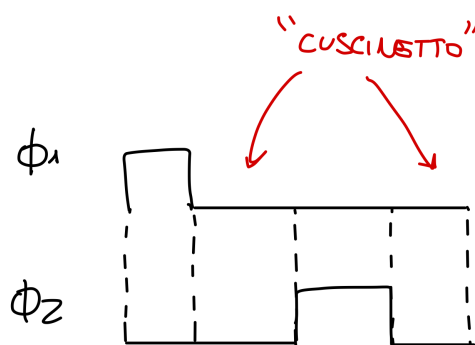


(b) Overlapping clock pairs

Il circuito avendo solo pass-transistor nMOS è sensibile solamente a una sovrapposizione di fase ovvero quella (1,1).

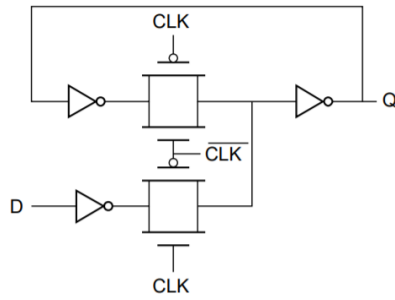
Quando ho una sovrapposizione (0,0) ho un funzionamento dinamico in cui i pass-transistor sono tutti spenti.

Per evitare la sovrapposizione di fase (1,1) posso pensare di usare due fasi distinte entrambe con duty cycle del 25% nel seguente modo

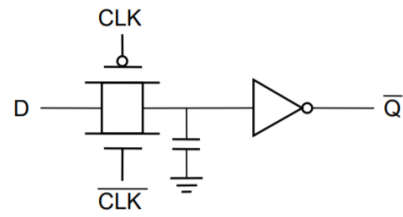


in questo modo se avessi un ritardo di fase ho un 25% del periodo di margine ("cuscinetto") prima di avere una sovrapposizione (1,1).

Meccanismi di storage dinamici



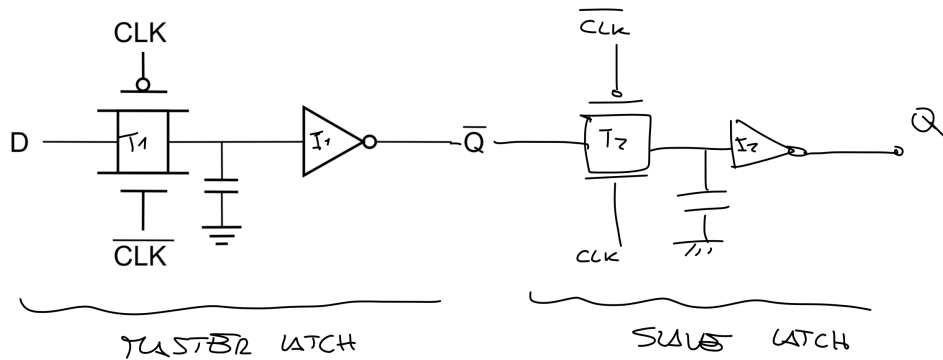
Static



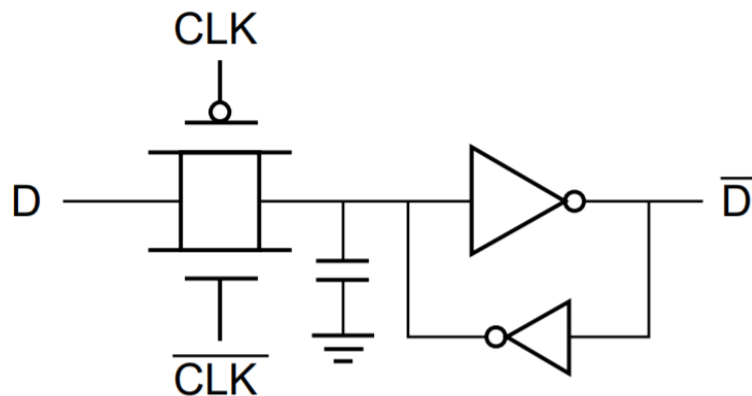
Dynamic (charge-based)

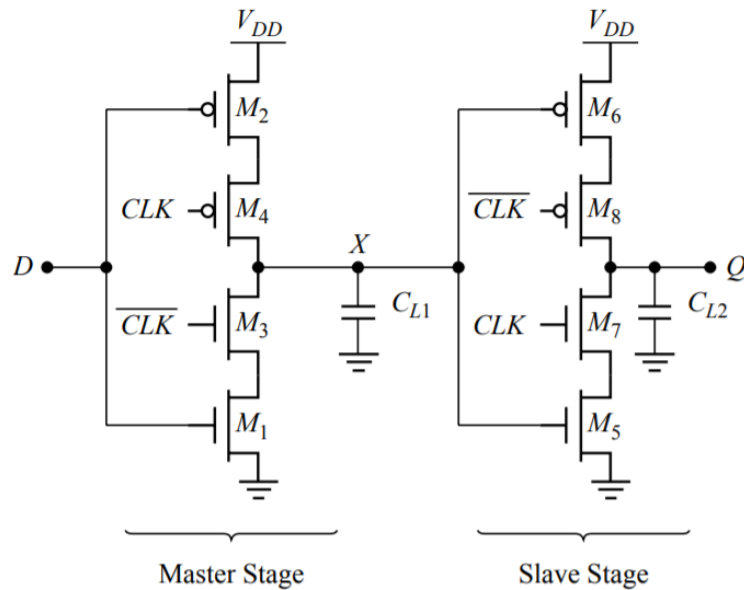
La memorizzazione dinamica si basa sulla carica e scarica di capacità quindi è necessario un refresh continuo in quanto si hanno delle correnti di leakage.

esempio di registro dinamico master-slave



Making a Dynamic Latch Pseudo-Static



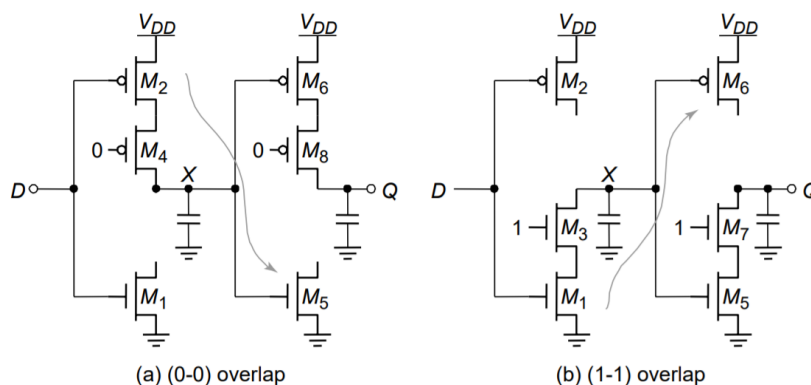


“Keepers” can be added to make circuit pseudo-static

- CLK = 0
 - M₃ e M₄ ON → il master si comporta come un inverter ed è trasparente
- CLK = 1
 - M₇ e M₈ ON → lo slave si comporta come un inverter ed è trasparente

Essendo la cascata di due inverter praticamente, il segnale si propaga solamente quando è attivo il PU di uno e il PD dell'altro o viceversa. Questo fa sì che sia molto più robusto alle sovrapposizioni dei clock

Insensitive to Clock-Overlap



Ad esempio nel caso (0,0), se nell'istante di campionamento del master c'è una sovrapposizione 0,0 e il dato commuta nell'istante di campionamento o poco dopo:

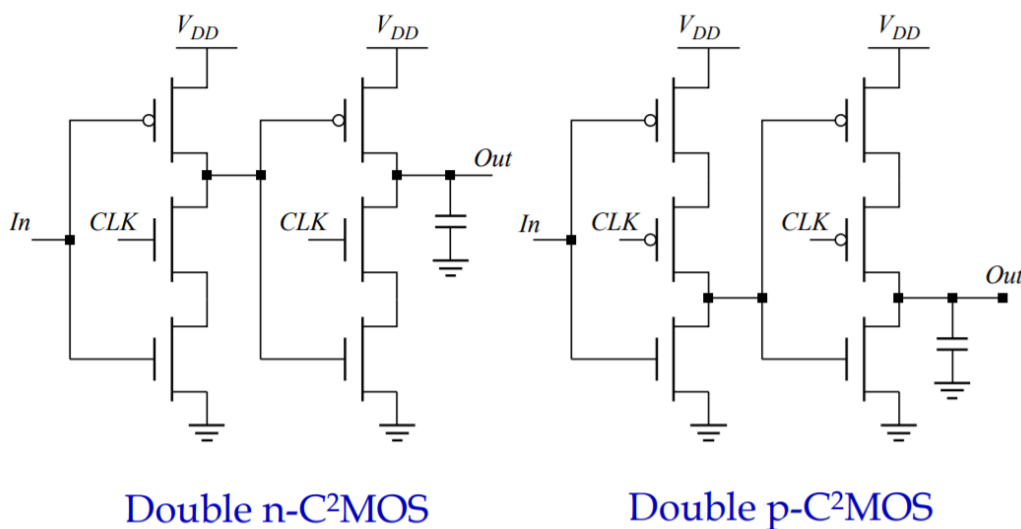
- il master può solamente catturare uno zero per accendere il PU
- il nodo x viene caricato a vdd (“1”)

- quindi va nel PD dello slave che però è staccato da Q (M_7 è spento con $\text{clk} = 0$) e quindi l'uscita non viene toccata
- quando le fasi si mettono a posto (cioè $\text{clk} = 0$ e $\text{!clk} = 1$) lo slave è in hold quindi non copia x in uscita: non c'è **corsa critica**

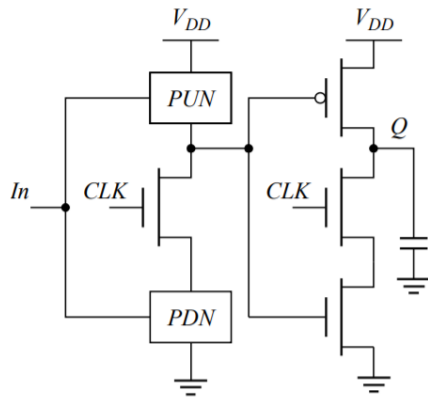
Quindi in questo circuito non c'è corsa critica (campionamento del dato sbagliato) e con l'introduzione di un tempo di hold è possibile anche evitare le sovrapposizioni di fase

l'unico caso critico è quello del fatto di avere delle transizioni dei clock lente. Infatti se per esempio il clock e il clock negato sono a $V_{DD}/2$ (nella transizione), tutti i p e n MOS sono parzialmente conduttivi. Per delle transizioni ragionevoli non dovrebbero esserci problemi di corse critiche

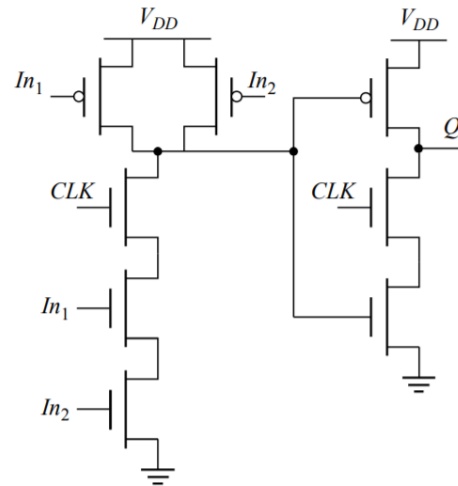
TPSC: truly single-phased latches



- **Il circuito non è influenzato dal ritardo dei clock**
- sono due latch che sono trasparenti rispettivamente quando $\text{clk} = 1$ e $\text{clk} = 0$ in cui ho due invertitori in serie



Example: logic inside the latch

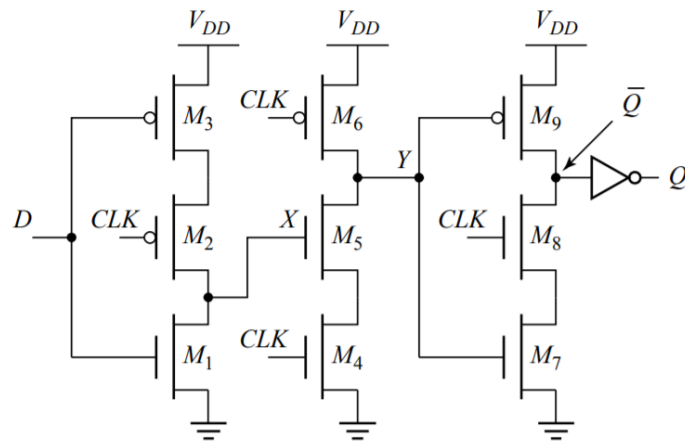


AND latch

- unisco il campionamento con la funzione logica (nell'esempio è nand + not)
- nell'esempio, se il $\text{clk} = 0$ il PD del master è spento, il PU porta il nodo x a V_{DD} ma lo slave può valutare solo uno zero (anche il suo PD è spento) quindi tutto il registro è in hold
- con $\text{clk} = 1$ ho la nand + la not

Precharged Register (TSPC: truly single-phased circuit)

Posso migliorare il numero di transistor



p-C²MOS - double n-C²MOS

- primo stadio $I_1 \rightarrow$ invertitore quando $\text{clk} = 0$
- terzo stadio $I_3 \rightarrow$ invertitore quando $\text{clk} = 1$
- secondo stadio $I_2 \rightarrow$ logica dinamica
 - $\text{clk} = 0 \rightarrow$ precarica del nodo y con mosfet M_6
 - $\text{clk} = 1 \rightarrow$ valutazione dell'ingresso accendendo M_4
- $\text{clk} = 0$
 - I_1 trasparente $\rightarrow$ porta in X !D
 - I_2 precarica $\rightarrow$ porta y a V_{dd}

- I_3 spento $\rightarrow$ y alta impedenza
- $clk = 1$
 - I_1 spento
 - I_2 valutazione
 - I_3 acceso $\rightarrow$ porta in Q y

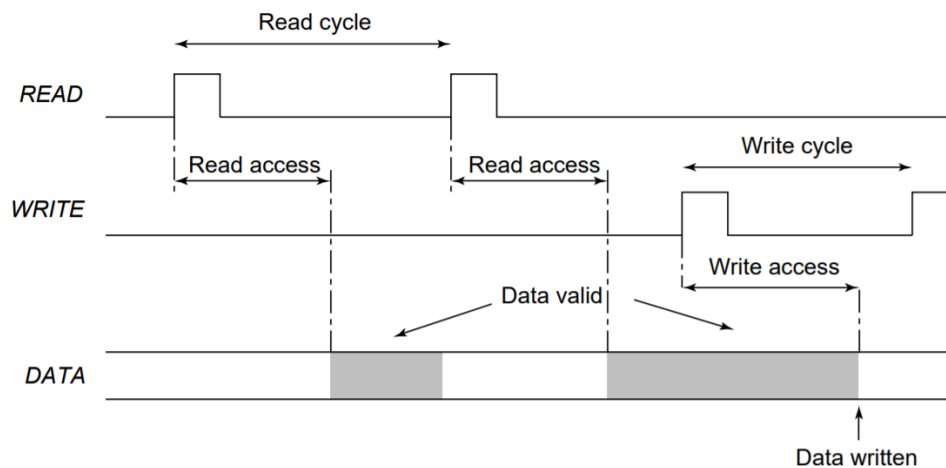
Il circuito è positive edge-triggered, e per un corretto campionamento è necessario un tempo di setup del dato (in modo che possa raggiungere il nodo x) che equivale di fatto al tempo di propagazione del primo invertitore.

Semiconductor Memories

classificazione delle memorie

- Read-Write Memories
 - Random Access (SRAM, DRAM)
 - Non-random access (FIFO, LIFO, Shift Register, CAM)
- Non-Volatile Read-Write memory (EPROM, E²PROM, FLASH)
- Read-only memory (ROM) (Mask-Programmed, programmable PROM)

Memory timing: Definizione dei tempi



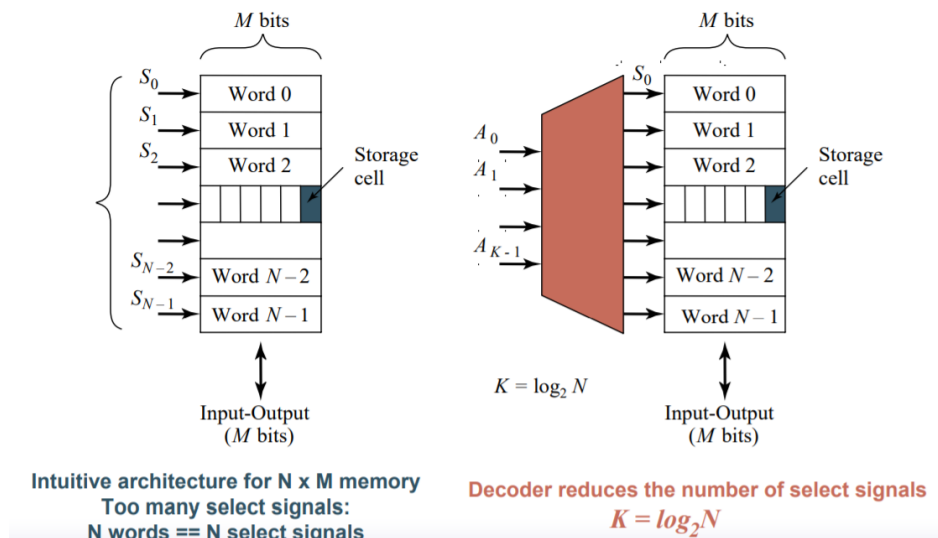
Read access → tempo dopo il quale, nel dato in uscita, trovo il dato letto (tempo per leggere il dato)

Read cycle → non si possono fare due letture consecutive troppo ravvicinate per la presenza di circuiti periferici che usano il dato letto pertanto è più lungo del read access

Write access → tempo per scrivere il dato

write cycle → tempo più lungo del write access

Memory Architecture: Decoders



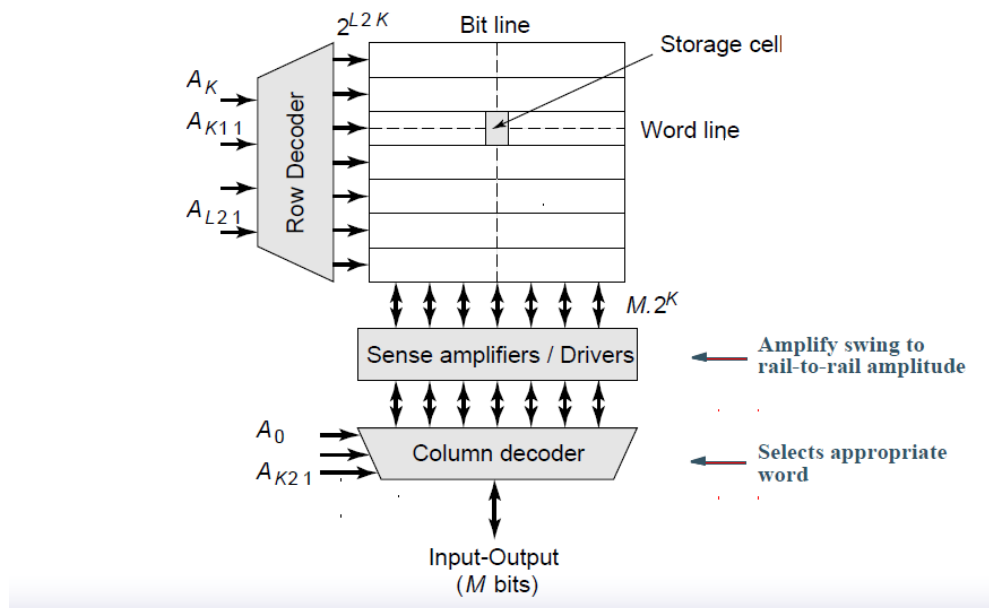
Solitamente il numero di parole immagazzinate N è molto più grande del numero di bit M usati per parola e se associassi una sequenza di bit a una specifica parola avrei N sequenze diverse e quindi N segnali di selezione

Si fa quindi uso dei decoder:

- $N = 2^k$ parole $\rightarrow k = \log_2 N$
- ho k segnali di selezione che sono molti di meno rispetto agli N di prima

Array-Structured Memory Architecture

Problem: ASPECT RATIO or HEIGHT $\gg$ WIDTH



- **word line:** linee che selezionano le parole e sono le uscite del decodificatore
- **bit line:** linee di input/output dei dati che uso per leggere o scrivere
- **storage cell (cella di memoria):** stanno nell'incrocio tra bit line e word line

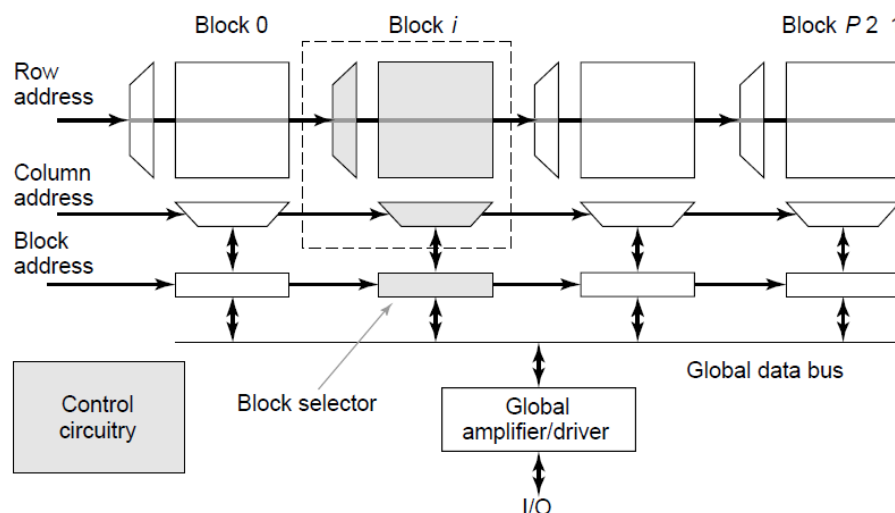
- **sense amplifier**
- **row decode**: decodificano gli indirizzi di riga ($L-K$ bit). E' di fatto un multiplexer perchè di fatto seleziona la riga
- **column decoder**: decodificano gli indirizzi di colonna (K bit). Dopo che il decodificatore di riga ha selezionato la riga, con la bit line porto in uscita l'effettivo dato che andrà scritto/letto.

Un indirizzo è quindi rappresentato da L bit ($(L-K) + K$). Al giorno d'oggi le memorie sono dell'ordine dei Giga quindi ogni indirizzo è rappresentato da almeno 30 bit.

il problema di questa organizzazione matriciale si pone quando il numero di parole è molto maggiore del numero di bit di una singola parola. Infatti quando devo far scorrere le informazioni lungo le **word line** o lungo le **bit line** uso delle interconnessioni on-chip le quali però all'aumentare della lunghezza fanno aumentare più che linearmente il ritardo (indicativamente quadratico).

Di conseguenza dato che il ritardo è dato dal caso peggiore, conviene fare delle memorie che siano per così dire "quadrate" ovvero che le interconnessioni orizzontali e verticali siano simili

Hierarchical Memory Architecture



Advantages:

1. Shorter wires within blocks
2. Block address activates only 1 block => power savings

Con l'aumentare delle dimensioni delle memorie la dimensione della matrice è diventata un problema soprattutto per la lunghezza delle interconnessioni. Si fa quindi uso di più matrici interconnesse tra loro. In questo caso gli indirizzi si spezzano ulteriormente, infatti ho

- un selettore del blocco
- un selettore della riga (per ogni blocco)
- un selettore della colonna (per ogni blocco)

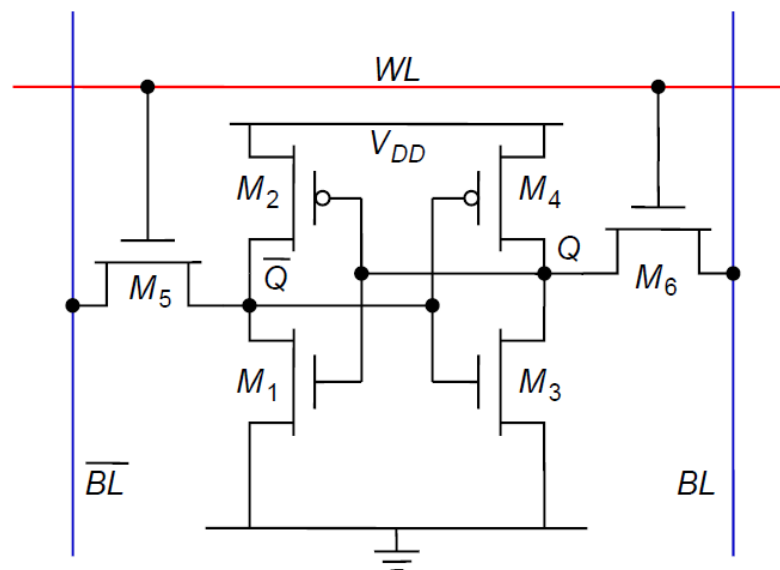
vantaggi:

- posso limitare la dimensione delle singole matrici (interconnessioni più corte)
- i circuiti periferici analogici come i sense amplifier consumano molto in termini di energia: avendo più blocchi posso attivare solamente i circuiti periferici legati al singolo blocco mentre tutti gli altri sono spenti

Read-Write Memories (RAM) (Volatile)

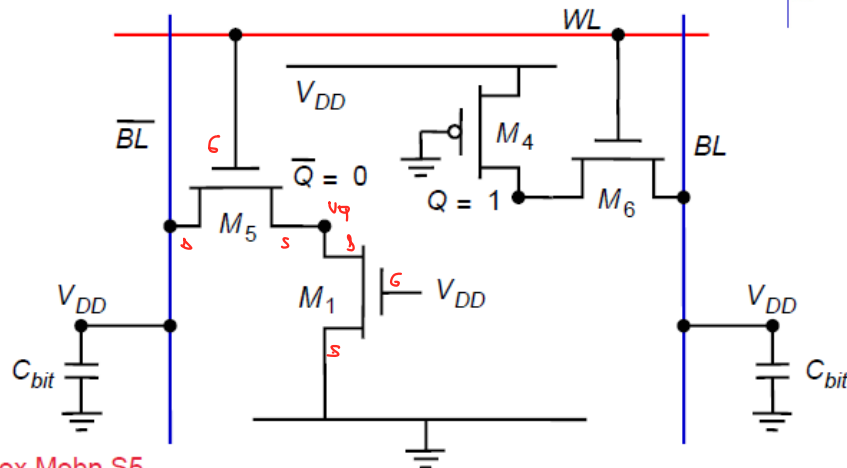
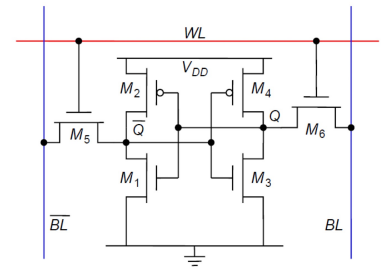
- SRAM (Static RAM)
 - è statica perchè non ha bisogno di refresh
 - i dati sono immagazzinati fintanto che c'è alimentazione (memoria volatile)
 - veloci
 - differenziali → la cella restituisce il dato in forma dritta e negata
 - grandi → sono composti da 6 transistor per cella
 - 2 inverter (bistabile)
 - 2 mos per leggere e scrivere
- DRAM (Dynamic RAM)
 - è necessario un refresh dei dati (sono immagazzinati in capacità)
 - più lente delle SRAM
 - single ended → la cella restituisce un solo dato (o dritto o negato)
 - piccole → composte da 1-3 transistor per cella
 - condensatore
 - 1 transistor di accesso
 - **il dato viene perso dopo essere stato letto**

6-transistor CMOS SRAM Cell



- WL (word line)
 - se bassa → la cella è in ritenzione (ferma)
 - se alta → accende M_5 e M_6 facendo passare le BL (bit line) che sono il dato da memorizzare

Lettura (read)



$k_{n,M5} = C_{ox} \mu_{n} S_5$

$k_{n,M1} = C_{ox} \mu_{n} S_1$

$CR = S_1/S_5$

$$k_{n,M5} \left((V_{DD} - \Delta V - V_{Tn}) V_{DSATn} - \frac{V_{DSATn}^2}{2} \right) = k_{n,M1} \left((V_{DD} - V_{Tn}) \Delta V - \frac{\Delta V^2}{2} \right)$$

$$\Delta V = \frac{V_{DSATn} + CR(V_{DD} - V_{Tn}) - \sqrt{V_{DSATn}^2(1 + CR) + CR^2(V_{DD} - V_{Tn})^2}}{CR}$$

$$I_{SAT} = \frac{\beta}{2} (V_{GS} - V_T)^2 \quad V_{DSAT} = V_{GS} - V_T$$

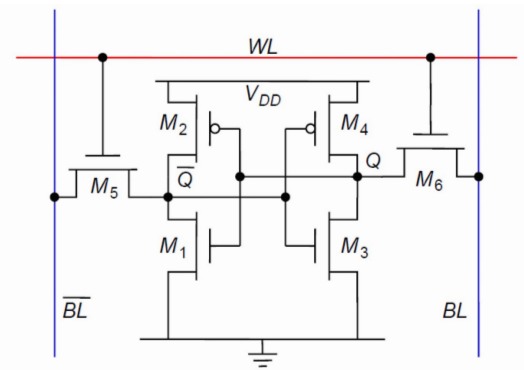
$$I_{SAT} = k \left[(V_{GS} - V_T) V_{DSAT} - \frac{V_{DSAT}^2}{2} \right]$$

- entrambe le BL sono precaricate a V_{DD}
 - sono in alta impedenza ovvero vengono staccate dal driver che le pilota
 - le bit line hanno una capacità grande data dalle interconnessioni lunghe e dai nodi (mosfet) che sono collegati alla BL
- supponiamo che nella cella sia immagazzinato un "1" ($Q = 1$)
 - vuol dire che in ingresso all'invertitore di destra c'è uno zero (è acceso solo M_4)
 - $!Q = 0$
 - in ingresso all'invertitore di sinistra c'è un 1 quindi è acceso solo M_1
- in generale però dove l'uscita è "0" (sia per Q che per $!Q$), vorrei che si producesse un effetto sulla BL che è vicina a quell'uscita. Nel nostro esempio la BL di sinistra
- In particolare vorremmo che la BL si scaricasse completamente a massa ma questo non sarà mai possibile in quanto:
 - i tempi sarebbero molto lunghi (alto read access time) in quanto la capacità della BL è molto grande (100 volte più grande di un transistor elementare)
 - se le scaricassi tutte, comunque ad ogni lettura dovrei ricaricarle fino a V_{DD} quindi sarebbe un'operazione energivora e lunga
- La BL quindi viene sbilanciata (ad esempio del 10% rispetto a V_{DD}) e questo sbilanciamento porta all'intervento dei **sense amplifier**: amplificatori differenziali che leggono le due BL e consentono la lettura del dato

problema:

quando attivo la WL (la porto a V_{DD}) il mosfet M_5 ha:

- drain a V_{DD}
- gate a V_{DD}
- source a 0 (!Q = 0)



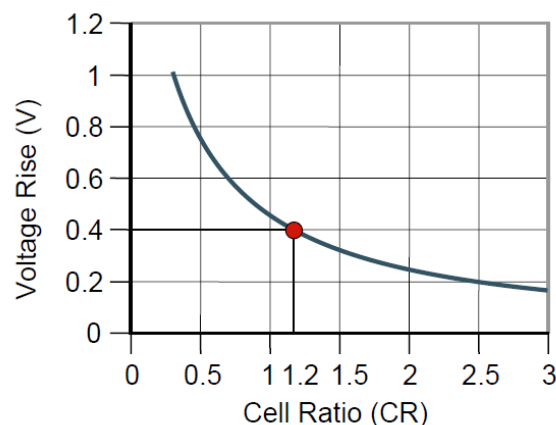
Di conseguenza il pass-transistor inietterà tutta la corrente sul source che vedrà il suo potenziale alzarsi

(**sovratensione**) e poi abbassarsi in quanto si scaricherà su M_1 .

Tuttavia !Q è l'ingresso dell'invertitore di destra quindi se la sovratensione è troppo ampia o troppo lunga si ha il problema del **read abset** ovvero viene cambiato il contenuto della cella.

Questo problema si verifica quando la sovratensione accende il transistore n (M_3) dell'invertitore di destra. Si progetta quindi la cella affinché questa sovratensione sia sufficientemente contenuta e in particolare minore della tensione di soglia del n mos

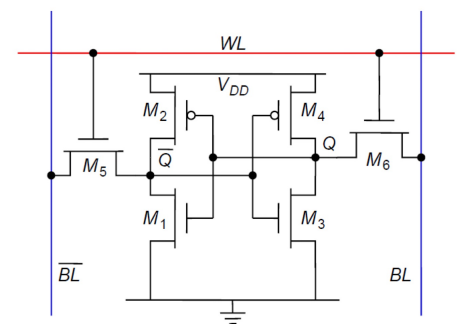
1. inserisco una capacità che posso dimensionare nel nodo !Q
2. attendo che il nodo !Q, a rapporto tra M_5 e M_1 , arrivi ad una tensione intermedia tra V_{DD} e massa
 - a. M_5 è saturo: se $V_q < V_t \rightarrow V_{dd} > V_{dd} - V_t$
 - b. M_1 è triodo: se $V_q < V_t \rightarrow V_q < V_{dd} - V_t$
 - c. ΔV : potenziale di Q
 - d. K è il beta
 - e. CR (cell ratio): rapporto di forza tra M_1 e M_5



$$CR = \frac{W_1/L_1}{W_5/L_5}$$

Più il rapporto di forza è grande (ovvero M_1 è più forte) minore sarà la sovratensione del nodo

Write



$K_n, M6 = C_{ox} \mu_{obn} S_6$
 $K_p, M4 = C_{ox} \mu_{obp} S_4$
 $PR = S_4/S_6$

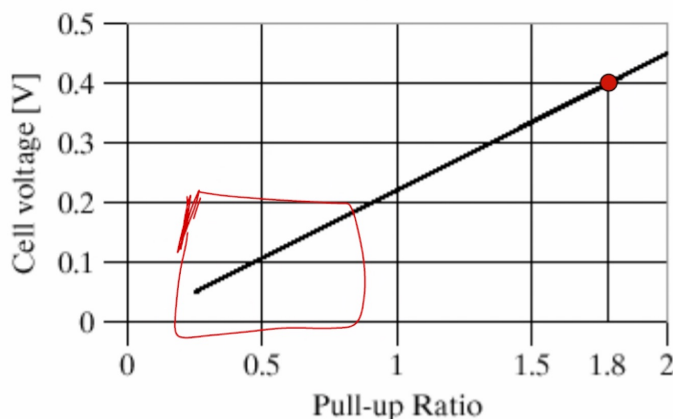
$$k_{n, M6} \left((V_{DD} - V_{Tn}) V_Q - \frac{V_Q^2}{2} \right) = k_{p, M4} \left((V_{DD} - |V_{Tp}|) V_{DSATp} - \frac{V_{DSATp}^2}{2} \right)$$

$$V_Q = V_{DD} - V_{Tn} - \sqrt{(V_{DD} - V_{Tn})^2 - 2 \frac{\mu_p}{\mu_n} PR \left((V_{DD} - |V_{Tp}|) V_{DSATp} - \frac{V_{DSATp}^2}{2} \right)}$$

- ora le BL portano il dato che bisogna scrivere nella cella (nell'esempio $BL = 0$ mentre $Q = 1$)
- le possibilità sono due:
 - o scrivo nella cella lo stesso dato
 - o scrivo il dato opposto
- nel caso di scrittura del dato opposto, la tensione di uscita è sempre opposta alla tensione della BL da tutte e due le parti
- nella lettura abbiamo detto che sul lato sinistro non posso scrivere su !Q in quanto la BL varierebbe di poco (la scarica nelle memorie, ecc.)
- quindi devo scrivere sul lato destro: lo si può fare solamente se il pass-transistor M_6 prevale su M_4 in modo che Q si possa scaricare (write ability)
 - non si scarica del tutto ma bisogna far sì che sia minore della soglia elettrica dei transistori del PD in modo che essi si spengano (mentre i p mos del PU si accendono così che in !Q venga scritto un 1)

quindi:

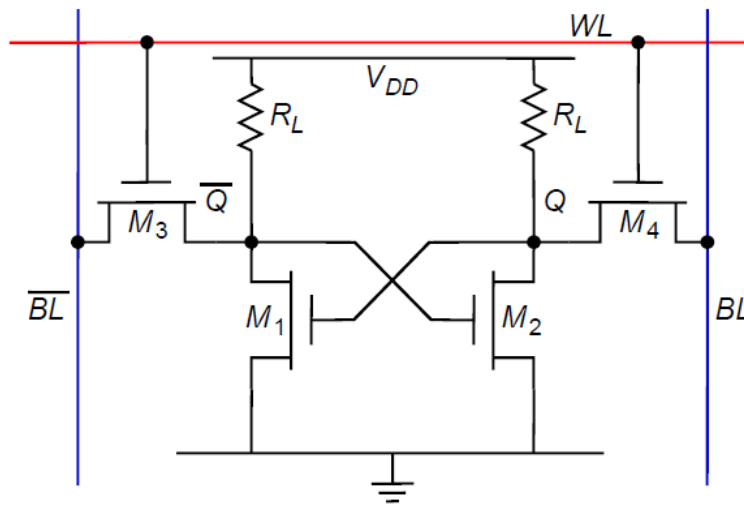
- M_6 triodo: noi vogliamo che $V_q < V_t$ quindi $V_d < V_g - V_t \rightarrow V_q < V_{dd} - V_t \rightarrow \text{circa } 0 < V_{dd} - 0 \rightarrow \text{triode}$
- M_4 saturo: $V_{dd} - V_q > V_{dd} - V_t \rightarrow \text{se } V_q < V_t \text{ è rispettata}$
- PR (pull-up ratio): rapporto di forza tra M_6 e M_4



$$PR = \frac{S_4}{S_6}$$

più è piccolo, più $V_q \rightarrow 0$
 (ovvero $S_6 \gg S_4$)

Resistance-load SRAM Cell



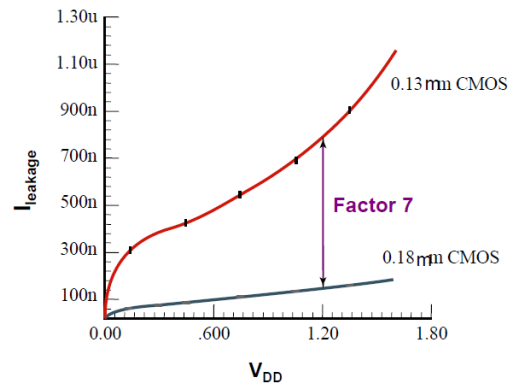
Static power dissipation -- Want R_L large
Bit lines precharged to V_{DD} to address t_p problem

- il PD diventa importante in lettura in quanto deve essere particolarmente forte
- Il PU deve essere debole per la scrittura (il pass-transistor deve essere più forte, vedi sopra)
 - non è necessario che sia fatto di transistor, in questo caso il PU è costituito da delle resistenze (polisilicio poco drogata)
 - consumo di area
- questo circuito non ha transistori p
- BL precaricate a V_{DD}
- quando gli nmos sono spenti la corrente delle R deve essere sufficientemente più grande della corrente di off dei mosfet per portare il nodo a V_{DD} . Basta una piccola corrente quindi possono essere anche resistenze elevate

Leakage nelle SRAM

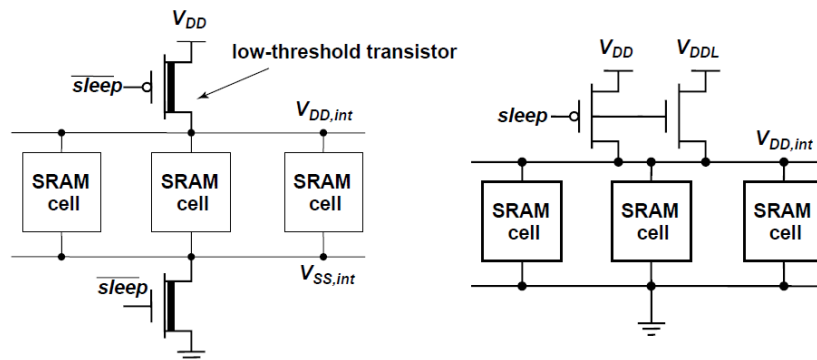
Con lo scaling della tecnologia si sono abbassate le tensioni di alimentazione e anche le tensioni di soglia dei transistori. Questo ha portato ad avere dei transistor che si spengono “più lentamente” e quindi ad avere delle correnti di leakage (o correnti di off) non proprio trascurabili portando ad uno **stand-by power**

Data Retention in SRAM



SRAM leakage increases with technology scaling

Suppressing Leakage in SRAM

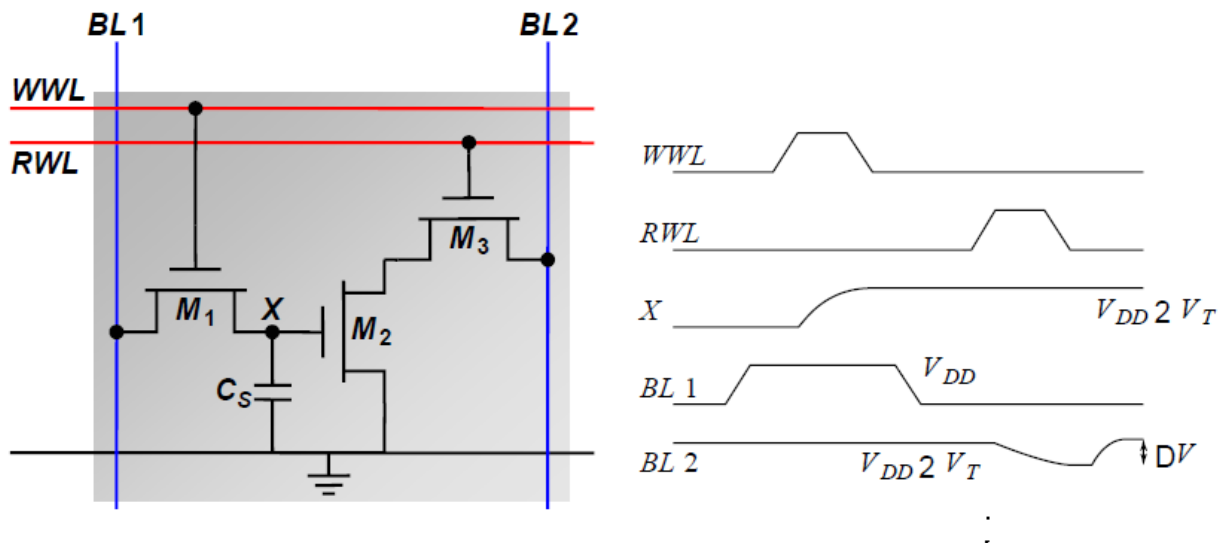


Inserting Extra Resistance

Reducing the supply voltage

- mettendo i transistori in serie al PU e PD comandati da un segnale di sleep che mi dice quando il circuito deve funzionare o è appunto in sleep mode

3-transistor DRAM Cell



No constraints on device ratios

Reads are non-destructive

Value stored at node X when writing a "1" = $V_{WWL} - V_{Tn}$

- C_S capacità di gate
- M₁, M₃ → mos di accesso rispettivamente di scrittura e lettura
- WWL : write word line
- RWL: read word line

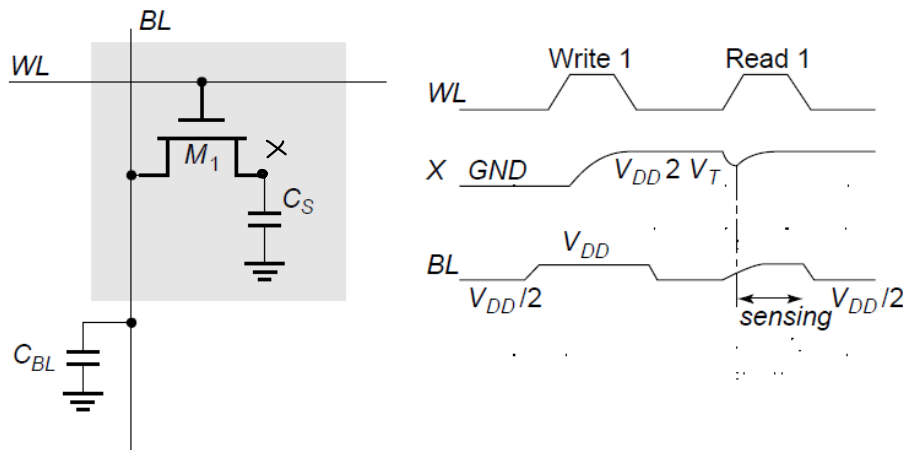
write

- WWL a V_{dd}, RWL a 0 (mutuamente esclusive)
- X = 0, voglio scrivere un 1 con BL1 = 1
- carico la capacità C_s fino a V_{dd}-V_t (pass-transistor con n mosfet)
 - potrei sovvertire con dei condensatori (oltre V_{dd}) le BL (che sono delle capacità) per avere appunto una carica fino a V_{dd}

read

- RWL a V_{dd}, WWL a 0
- X = 1, voglio leggerlo
- la BL2 inizialmente precaricata a V_{dd} o V_{dd}-V_t si scarica se su X c'è un 1 (che accende M2 che è collegato a massa)
 - se X era 0 allora non si scaricava
- la lettura non è distruttiva

1-Transistor DRAM Cell



Write: C_S is charged or discharged by asserting WL and BL.

Read: Charge redistribution takes place between bit line and storage capacitance

$$\Delta V = V_{BL} - V_{PRE} = V_{BIT} - V_{PRE} \frac{C_S}{C_S + C_{BL}}$$

Voltage swing is small; typically around 250 mV.

- precarico la BL a $V_{DD}/2$

write

- WL a V_{DD}
- BL a V_{DD}
- Carico C_S

read

- avviene per ripartizione di carica
 - distruttiva
 - meno robusta
 - cella molto piccola (vantaggio)
- BL precaricata a $V_{DD}/2$
- M_1 ha come source e drain due condensatori a tensioni diverse quindi si innesca un transitorio che si può esaurire se
 - M_1 si spegne (non è possibile in quanto il gate è a V_{DD} e il drain è a $V_{DD}/2$ e il source non riuscirà mai a raggiungere $V_{DD} - V_t$ per spegnere il transistor)
 - la V_{DS} si annulla e quindi annulla la corrente (le due capacità hanno la stessa tensione)
 - se X era caricata a V_{DD} la ripartizione porterà sulla BL una tensione più vicina a V_{DD} e viceversa se X era a 0 la ripartizione porterà sulla BL una tensione più vicina a massa leggendo così la x
 - va letta la differenza tra il valore precaricato della BL e il valore della BL

$$C_{BL} \cdot V_{PRE} + C_S \cdot V_{BIT} = V_{BL} (C_{BL} + C_S)$$

$$V_{BL} = \frac{C_{BL}}{C_{BL} + C_S} \cdot V_{PRE} + \frac{C_S}{C_{BL} + C_S} \cdot V_{BIT}$$

tensione di precharge della BL

tensione x

$$\begin{aligned} \Delta V &= V_{BL} - V_{PRE} \\ &= V_{PRE} \left(\frac{C_{BL} - C_{BL} - C_S}{C_{BL} + C_S} \right) + \frac{C_S}{C_{BL} + C_S} V_{BIT} \\ &= (V_{BIT} - V_{PRE}) \frac{C_S}{C_{BL} + C_S} \end{aligned}$$

- V_{pre} : tensione della BL precaricata
- V_{bit} : tensione della capacità C_S
- V_{bl} : tensione della BL a transitorio esaurito che sarà uguale a quella sulla capacità C_S
- il termine $\frac{C_S}{C_{bl} + C_S} = \frac{C_S/C_{bl}}{1 + C_S/C_{bl}} \approx \frac{C_S}{C_{bl}}$
 - essendo C_{bl} grande è un rapporto piccolo

Quindi:

- 1T DRAM richiede di avere un sense amplifier per ogni bit line che legga la differenza di tensione che si è innescata in lettura
- sono celle single ended (una sola uscita)
- la capacità C_S è inclusa esplicitamente (nel 3T DRAM era la capacità del gate di M2)
- la lettura è distruttiva: dopo ogni lettura è necessario un refresh
 - viene fatto dal sense amplifier che ripilota la BL così come l'ha letta per poterla refreshare

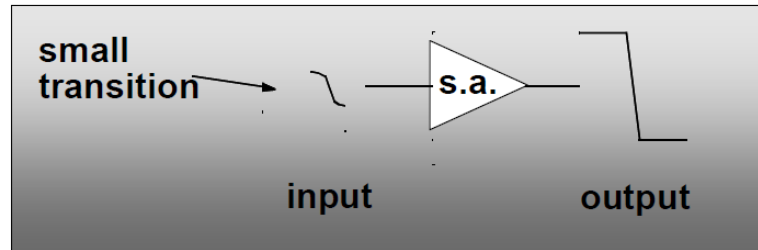
Sense Amplifier

$$t_p = \frac{C \cdot \Delta V}{I_{av}}$$

Annotations:

- make ΔV as small as possible (points to ΔV)
- large (points to I_{av})
- small (points to C)

Idea: Use Sense Amplifier



- $C \rightarrow$ capacità della bit line
- $\Delta V \rightarrow$ differenza di potenziale da leggere
- $I_{av} \rightarrow$ corrente media che serve per spostare la carica $C \cdot \Delta V$
- $T_p \rightarrow$ ritardo

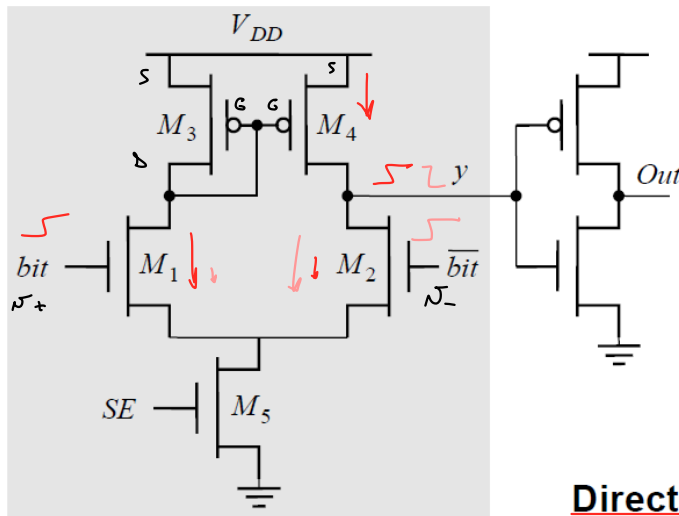
I sense amplifier sono circuiti analogici periferici che interpretano il dato che arriva dalla memoria. Essi non ottengono degli zeri o uni come i registri ma ottengono delle frazioni di V_{dd} .

Si usano i sense amplifier in quanto è preferibile che le escursioni delle BL siano il più piccole possibile. se così non fosse

- i tempi sarebbero lunghi
- si richiederebbe parecchia energia per caricare le capacità delle BL

Oggigiorno si possono leggere un centinaio di mV con i sense amplifier (le tensioni di V_{dd} sono dell'ordine di 1V)

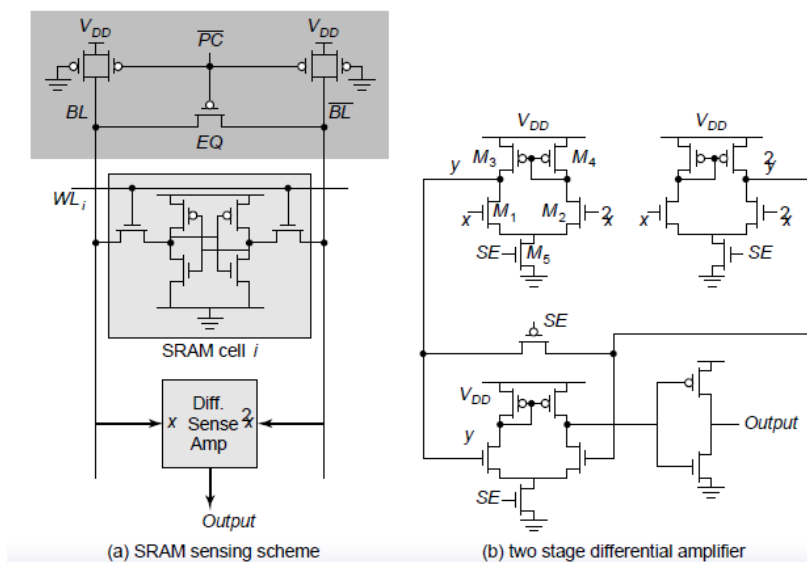
Differential Sense Amplifier (SRAT)



Directly applicable to SRAMs

- Il segnale di sensing SE serve per evitare che il circuito abbia un consumo statico di potenza (i circuiti analogici soffrono di questo problema)
 - viene attivato dopo che è stato generato quel deltaV da leggere
- bit → bit line (precaricate a Vdd nel caso delle SRAM)
- Se nel circuito bit e !bit sono uguali e i mosfet sono tutti uguali allora c'è un bilanciamento nel senso che y è uguale al drain del mosfet con bit (sx)
- quando c'è uno sbilanciamento, comincia a scorrere più corrente su uno dei due mosfet e quindi il nodo y si alza o si abbassa

Differential Sensing – SRAM



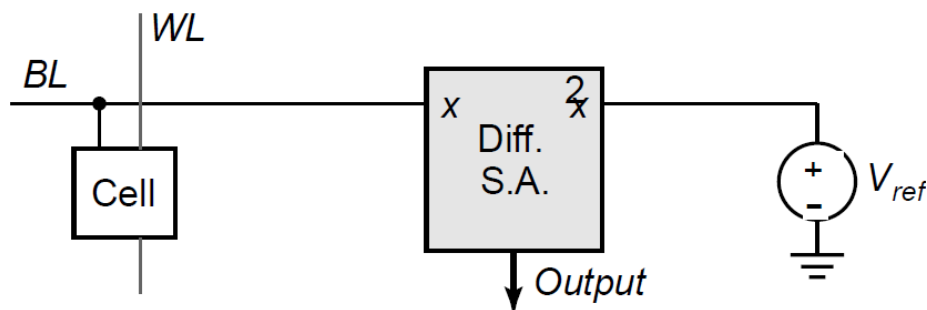
- IPC → transistore di precarica
 - quando è basso accende i transistori di precarica delle due BL + il transistore di equalizzazione EQ
 - il transistore EQ serve a bilanciare le due BL (tende a portare la sua Vds = 0)

- Dopo la precarica disattivo !PC e attivo la WL per la lettura
 - la cella che ha uno zero tende ad abbassare la BL dalla sua parte
- quando la BL si sbilancia abbastanza (si è generato un ΔV), attivo il sense amplifier

Single to differential conversion (DRAM)

Nella DRAM viene prodotto uno sbilanciamento sulla BL e lo voglio confrontare con $V_{dd}/2$ che era il valore di precarica della BL (così) so che dato è stato letto)

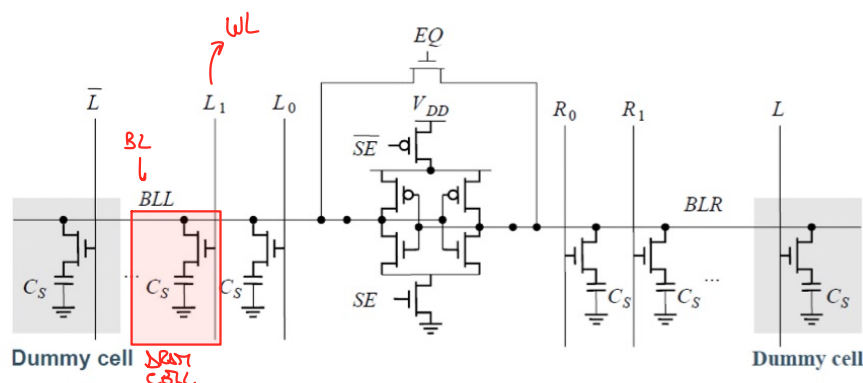
1. una possibilità è prendere un generatore di riferimento che però è critica: quel riferimento dovrebbe seguire le vicende delle celle in termini di
 - a. variazioni istante per istante di V_{dd}
 - b. variazioni di temperatura
 - c. problema dell'aging (i transistori invecchiano e possono cambiare le letture dopo moltissime letture)



How to make a good V_{ref} ?

2. in realtà si usa una cella di riferimento "dummy cell" che non contiene informazioni ma viene usata come cella di confronto per quelle che contengono informazione

Open bitline architecture with dummy cells



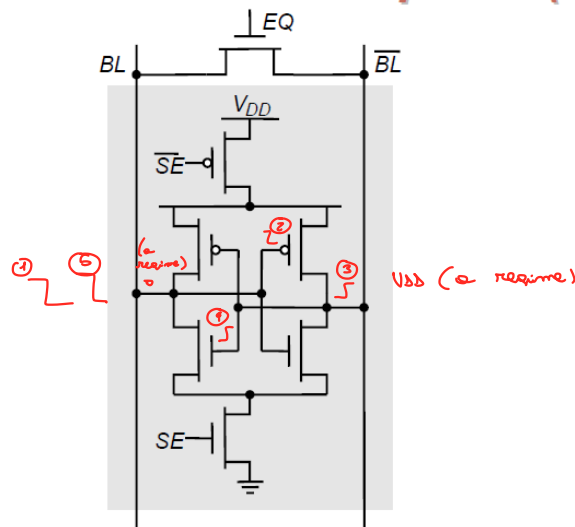
Consistono nel generare una colonna di celle che non vengono usate per immagazzinare informazione.

Dato che nella DRAM ho una sola BL per avere un segnale differenziale devo averne un'altra

- ho due BL: quella di sx (BLL) e quella di dx (BLR)
- quando $EQ = 1$ porto entrambe le BL a $V_{dd}/2$ (le dummy cell e tutte le altre celle sono precaricate a $V_{dd}/2$)
- Le WL di attivazione delle dummy cell sono !L e L
 - quando leggo una cella a sinistra (ovvero quella che ha il dato da leggere) attivo la dummy cell di destra e viceversa in modo che il sense amplifier abbia una BL a sx e una a dx

Il fatto di avere due dummy cell è per far sì che il tutto sia simmetrico

Latch-based sense amplifier (DRAM)



Initialized in its meta-stable point with EQ

Once adequate voltage gap created, sense amp enabled with SE

Positive feedback quickly forces output to a stable operating point.

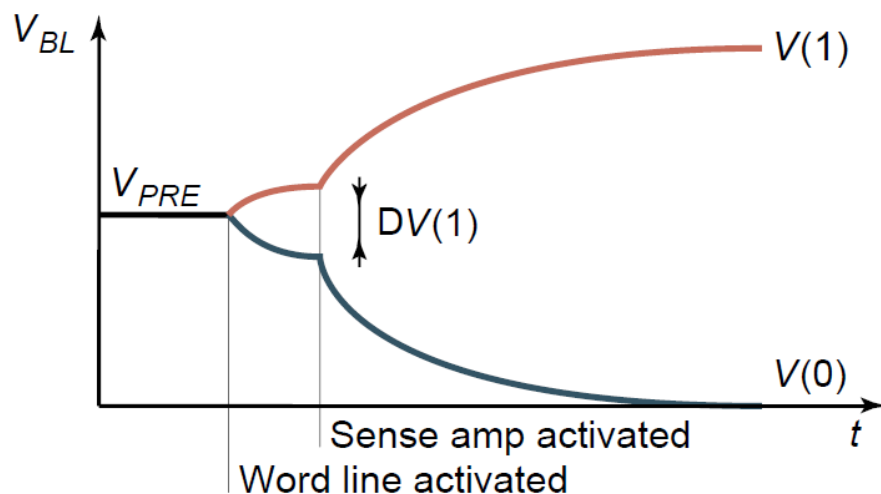
Nella DRAM la lettura è distruttiva: devo cercare un metodo per poter riscrivere il valore letto. Viene preso un bistabile e portato (con EQ) nel punto metastabile (idealmente $V_{DD}/2$ se gli invertitori sono perfettamente simmetrici) che ha tanto guadagno e lì non sta mai (è un punto instabile).

Quando $SE = 0$ e $!SE = 1$ posso forzare il circuito nello stato $V_{DD}/2$ in tutte e due le uscite (valore a cui precarico le BL).

Prima di attivarlo (ovvero $SE=1$ e $!SE = 0$) lascio il tempo alle BL di sbilanciarsi un pò in modo che poi il bistabile vada a finire in uno dei punti stabili.

- Le BL dettano in primo luogo lo sbilanciamento al sense amplifier (ingresso)
- successivamente il sense amplifier ripilota le BL portandole ai valori stabili
- se la WL è rimasta attiva riscrivo il dato nella cella con le BL
 - quindi il refresh del dato avviene semplicemente leggendo

Sense Amp Operation



Row Decoders

Collection of 2^M complex logic gates Organized in regular and dense fashion

(N)AND Decoder

$$WL_0 = A_0 A_1 A_2 A_3 A_4 A_5 A_6 A_7 A_8 A_9$$

$$WL_{511} = \bar{A}_0 \bar{A}_1 \bar{A}_2 \bar{A}_3 \bar{A}_4 \bar{A}_5 \bar{A}_6 \bar{A}_7 \bar{A}_8 \bar{A}_9$$

M : n° di bit

NOR Decoder

$$WL_0 = \overline{A_0 + A_1 + A_2 + A_3 + A_4 + A_5 + A_6 + A_7 + A_8 + A_9}$$

$$WL_{511} = \overline{A_0 + \bar{A}_1 + \bar{A}_2 + \bar{A}_3 + \bar{A}_4 + \bar{A}_5 + \bar{A}_6 + \bar{A}_7 + \bar{A}_8 + \bar{A}_9}$$

Il numero di transistor (**transistor count**) è $N_t = 2^M * (2 * M) = M * 2^{M+1}$

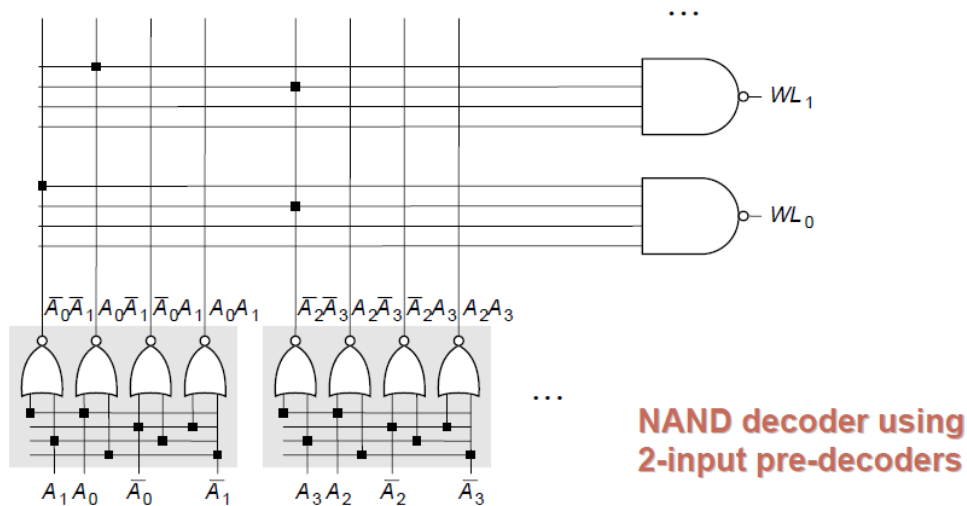
- ho 2^M gate (uscite) e ogni gate CMOS è costituito da $2 * M$ transistor

Sappiamo tuttavia che i gate CMOS cominciano a diventare lenti e inefficienti (nonchè ad occupare parecchia area) per $M > 4$

2^M : n° di uscite
punti di gate
 $2M$: n° di transistor
per gate

Decoder gerarchici

Multi-stage implementation improves performance

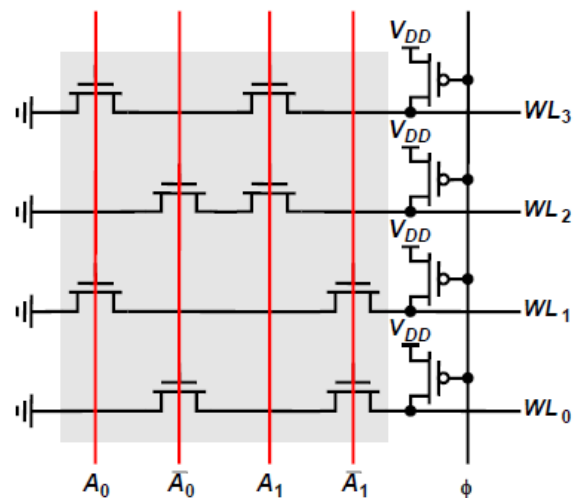
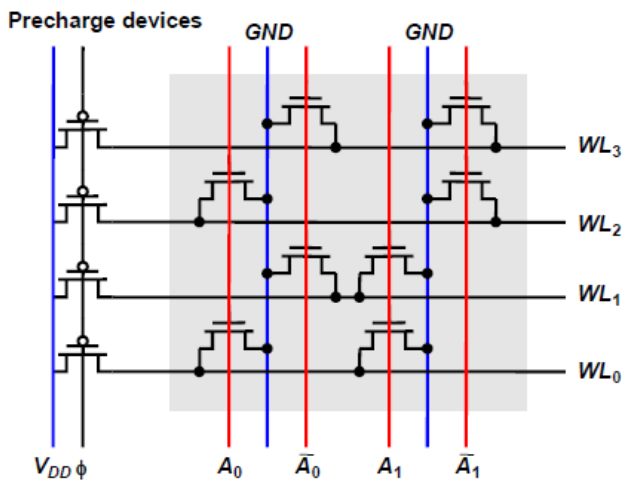


- $M = 8$
- logica a due stadi

$$\bar{A}_0 \bar{A}_1 \bar{A}_2 \dots \bar{A}_7 = (\overline{A_0 + A_1}) (\overline{A_2 + A_3}) (\overline{A_4 + A_5}) (\overline{A_6 + A_7})$$

- $N_t = 2^M * 8 + 2M * 4$
 - le nand sono 2^M che sono le uscite totali e ogni NAND a quattro ingressi ha 8 transistor
 - le NOR sono 16 e ogni NOR a due ingressi ha 4 transistor
 - sono circa la metà del modello precedente
- in questo modo le WL sono più leggere e aumentano quindi le prestazioni

Decoder dinamici



2-input NOR decoder

2-input NAND decoder

- elimina la ridondanza tra PU e PD dei gate
- consente un **transistor count** migliore
- NOR
 - Si precaricano le WL
 - Ho N connessioni di transistor collegati a massa in parallelo (N è il numero di bit)
 - è attivata solamente una WL per volta; in questo esempio solamente se A_0 e A_1 sono uguali a zero, la WL0 resta precaricata a Vdd e tutte le altre si scaricano essendo i mos in parallelo
 - dal punto di vista energetico è poco efficiente perchè precarico tutte le WL e le scarico tutte tranne una
- NAND
 - precarica tutte le WL
 - i transistor ora sono in serie
 - mantiene a Vdd tutte le WL eccetto quella con entrambi gli ingressi a uno che viene scaricata a massa
 - più efficiente
 - all'aumentare del numero di bit metto transistor in serie quindi dal punto di vista delle prestazioni dinamiche è peggiore

NOR

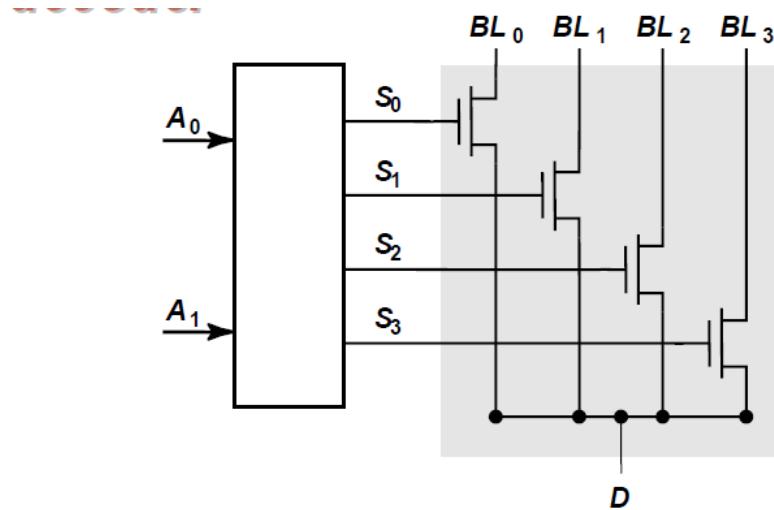
0	0	1
0	1	0
1	0	0
1	1	0

NAND

0	0	1
0	1	1
1	0	1
1	1	0

Column decoder

4-input pass-transistor based column decoder



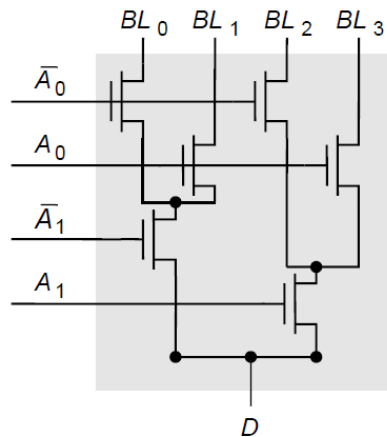
Advantages: speed (t_{pd} does not add to overall memory access time)

Only one extra transistor in signal path

Disadvantage: Large transistor count

- Di fatto la selezione delle colonne è una operazione di multiplexing
- in questo caso il decodificatore di colonna ha un numero di transistori pari alle BL
- se L è il numero di bit ho i transistori del decoder di riga + 2^L transistor
- il tempo per la decodifica di riga, lo sbilanciamento delle BL e l'arrivo ai sense amplifier, può essere sfruttato dal decodificatore di colonna (non introduce quindi ritardo al memory access time)

4-to-1 tree based column decoder



Number of devices drastically reduced

Delay increases quadratically with # of sections; prohibitive for large decoders

Solutions: buffers

progressive sizing

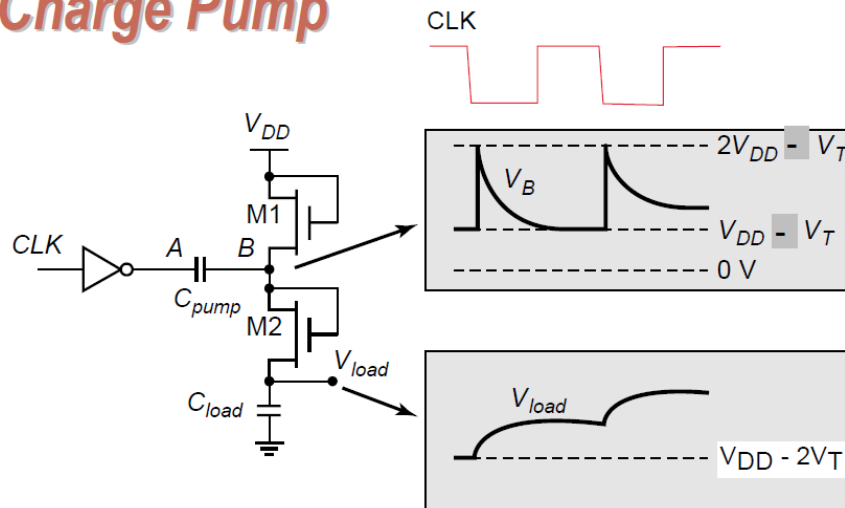
combination of tree and pass transistor approaches

- partendo dal basso (vicino a D) sul nodo di uscita si hanno 2 transistori, su quello più a monte (sopra A1) se ne hanno 4, ecc. (**struttura ad albero**)

- se ci fossero stati altri bit i transistor duplicavano ad ogni livello
- $N_t = 2 + 4 + \dots 2^L = 2*(2^L - 1)$
 - vantaggio nel numero di transistori
- il ritardo tende in modo quadratico

Charge Pump

Charge Pump

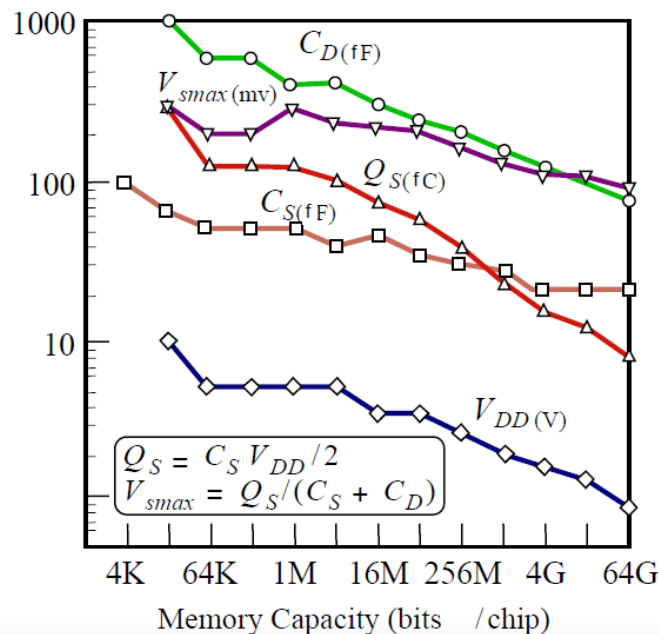


- Per far fronte al problema del trasferimento di tensioni che sono minori di Vdd si fa uso di questi circuiti di pump che fanno uso di un condensatore di pump
- M1 e M2 hanno connessione a diodo
- quando $clk = 1$
 - il nodo A è basso
 - pensando che Cload sia inizialmente scarico
 - il nodo B si carica a $V_{dd} - V_t$ (pass-transistor con solo nMos)
 - Vload si carica fino a $(V_{dd} - V_t) - V_t = V_{dd} - 2V_t = V_{load}$
 - sul condensatore alla fine c'è $C_{pump} (V_{dd} - V_t) \rightarrow B$ è $V_{dd} - V_t$ e $A = 0$
- quando $clk = 0$
 - A va ~~alto~~
 - in prima istanza il condensatore Cpump non cambia la sua carica quindi A va a V_{DD} e B va a $V_{DD} + (V_{DD} - V_t) = 2V_{DD} - V_t = V_B$
 - M_1 è spento in quanto il suo source ora è sopra Vdd quindi è in inversa (connessione a diodo) $V_{load} = V_{S2} > V_B = V_B$
 - M_2 è acceso (diodo in diretta) $\rightarrow$ carica il condensatore finchè può (perchè è un condensatore anche Cpump quindi c'è una sorta di ripartizione di carica)
 - il transitorio dura fintanto che B non raggiunge nuovamente $V_{DD} - V_t$

Il ciclo ripartirà facendo salire di carica e quindi di tensione C_{LOAD} un pò alla volta fintanto che la tensione Vload non sarà $(2V_{DD} - V_t) - V_t = 2(V_{DD} - V_t)$ ovvero la tensione per cui il diodo non sarà più acceso perchè in inversa (Vb arriva massimo a $2V_{DD} - V_t$.

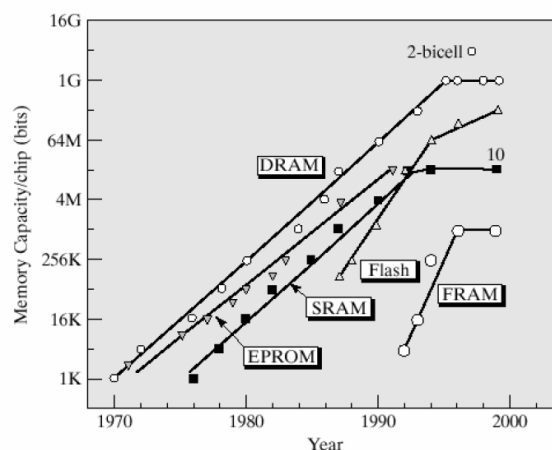
Si genera una tensione alta, ma la capacità di erogare corrente è limitata da Cpump e dalla frequenza di clock

Sensing-Parameters in DRAM



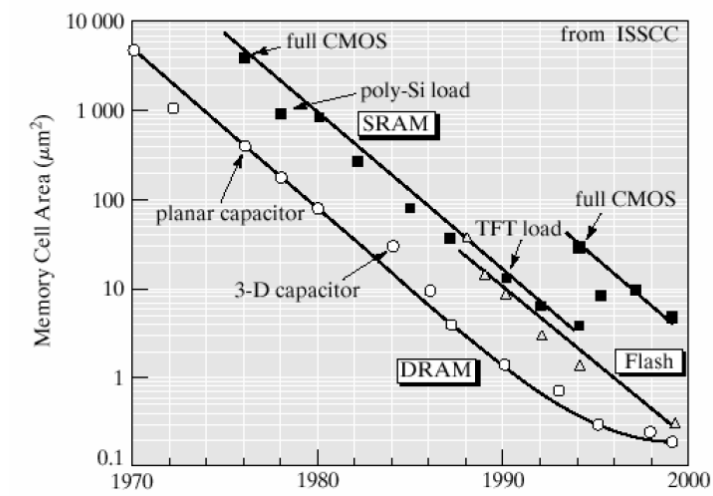
- sulle ascisse ci sono numero di bit per chip
- sulle ordinate ci sono scale logarithmiche
- C_S è la capacità di immagazzinamento
- C_D capacità della bitline
- V_{smax} è la tensione equivalente letta dal sense amplifier
- Si vede che Q_S si è ridotta più di C_S in quando si è ridotta V_{DD} . V_{DD} si è ridotta grazie allo scaling arrivando ai giorni nostri a circa 1V.
- la tensione di sbilanciamento della BL è data da $Q_S / (C_S + C_D)$

Semiconductor Memory Trends (updated)



- l'asse della capacità è logaritmico quindi la capacità delle memorie è cresciuta esponenzialmente

Trends in Memory Cell Area

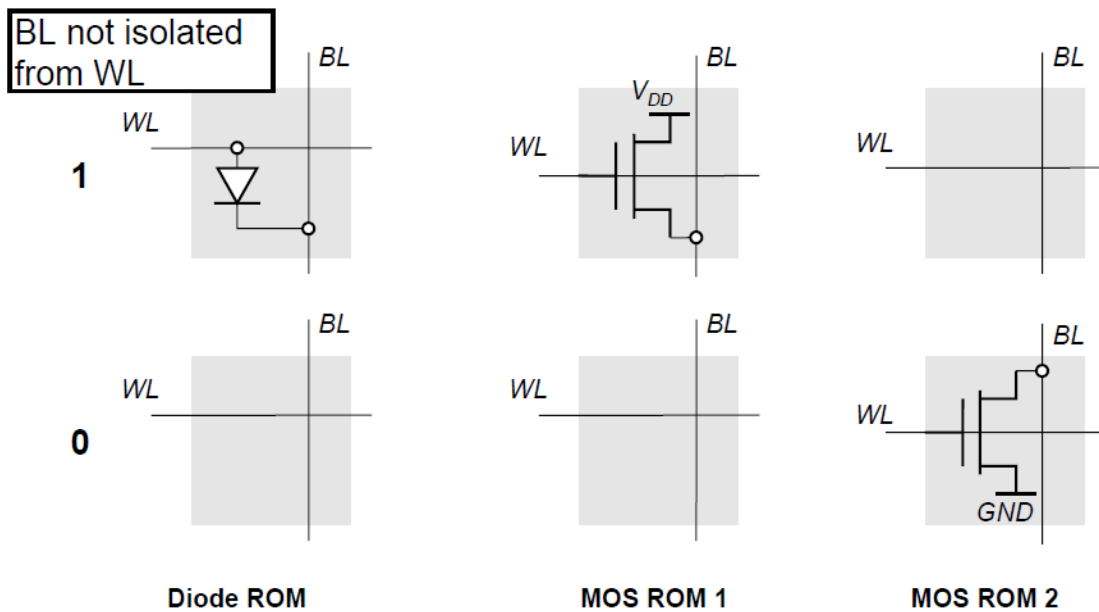


- le flash si sono sviluppate in 3D pertanto il numero di bit per area è aumentato (conta la planarità)

Memorie non volatili (ROM: Read-Only Memory)

L'informazione rimane disponibile anche quando non viene data alimentazione al sistema. All'inizio venivano create già col contenuto che potevano solamente essere letto.

Read-Only Memory Cells



ROM DIODO:

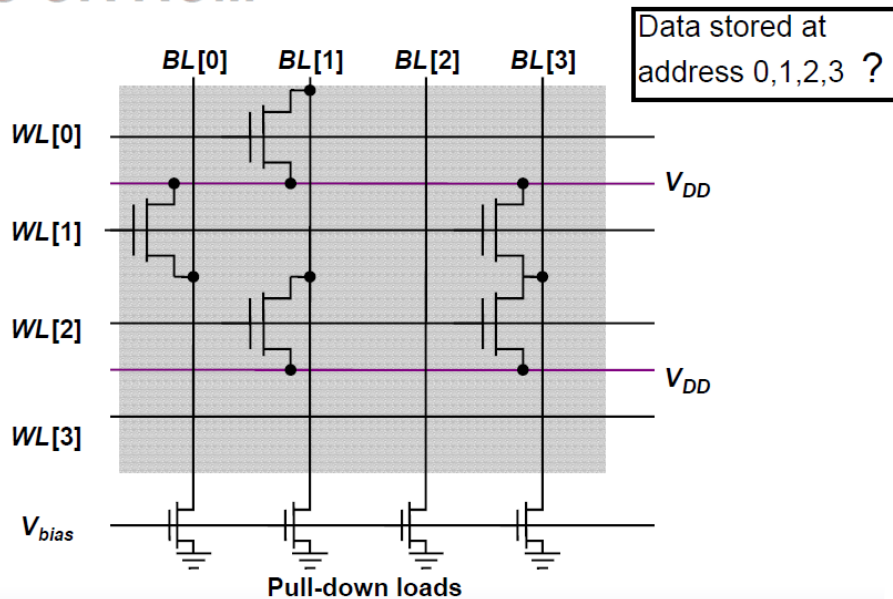
- diodo connesso tra WL e BL (non sono isolate come nelle non volatili)
- quando la WL si attiva inietta una corrente sulla BL
- Lo stato 1 o 0 corrisponde ad avere o non avere il diodo

ROM MOS

- il mos connette la BL
- le BL e WL sono isolate
- se attivo il mos carica la BL a 1 altrimenti rimane a 0 (primo caso)
- ho precaricato la BL a V_{DD} → se c'è il mos la scarico a 0 altrimenti no (secondo caso)
- leggo il contenuto ma non lo posso modificare

MOS OR ROM

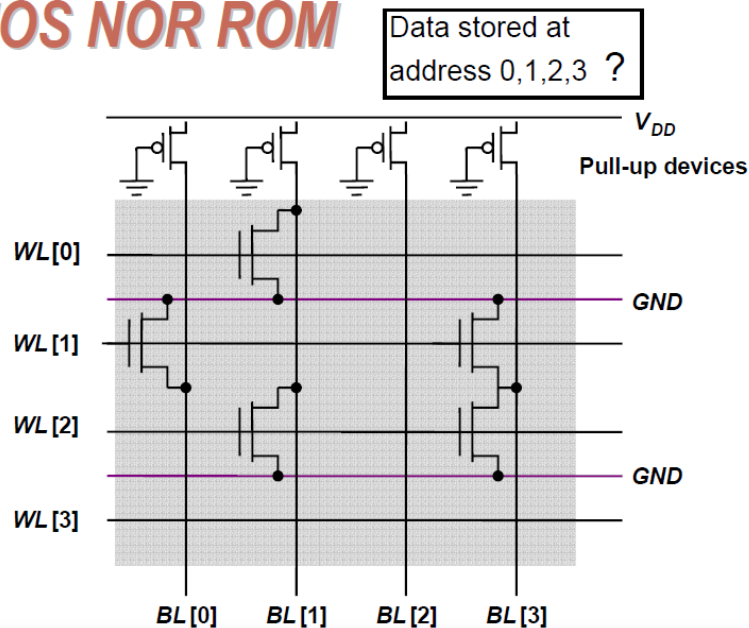
MOS OR ROM



- tiene le BL a 0 (prescaricate)
- non appena si accende una WL, le BL associate vengono caricate a Vdd ("1")
- il contatto (connessione tra due linee indicato col puntino nero) prende un'area paragonabile a quella del transistor (corrisponde al dato)
- nelle memorie l'eliminazione dei contatti riduce l'area della memoria (obiettivo di progetto)
- vdd non è presente tra WL1 e WL2 in quanto fisicamente le linee di Vdd sono separate e le celle pari e dispari stanno "dentro due Vdd" e condividono il contatto (in quella banda i mosfet se ci sono condividono il contatto. in questo modo si risparmia un contatto (quello condiviso è solo 1))
- architettura di tipo OR → i transistori che connettono una BL a Vdd sono in parallelo

MOS NOR ROM

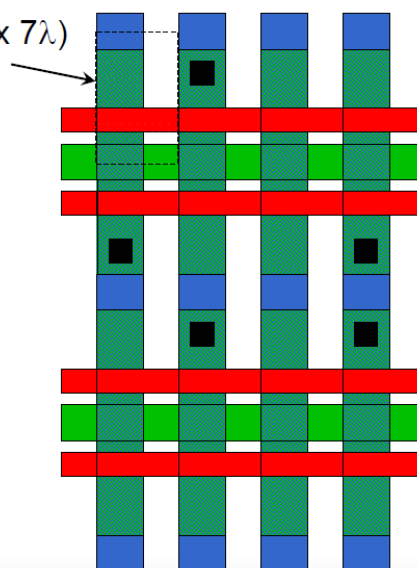
MOS NOR ROM



- architettura NOR → i transistori che connettono la BL a massa sono in parallelo

MOS NOR ROM Layout

Cell ($11\lambda \times 7\lambda$)



A large part of the cell is devoted to contacts to bitline and GND

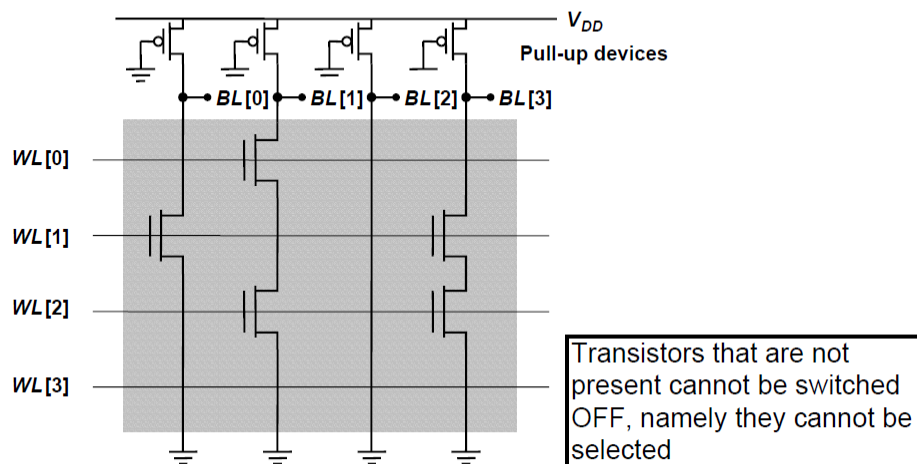
Programming using the Contact Layer Only

- Polysilicon
- Metal1
- Diffusion
- Metal1 on Diffusion

- le diffusioni corrispondono a gnd
- le connessioni di polisilicio sono le WL
- tra due linee di massa ci sono due WL
- in verticale ci sono le metal 1
- i quadratini neri sono i contatti

MOS NAND ROM

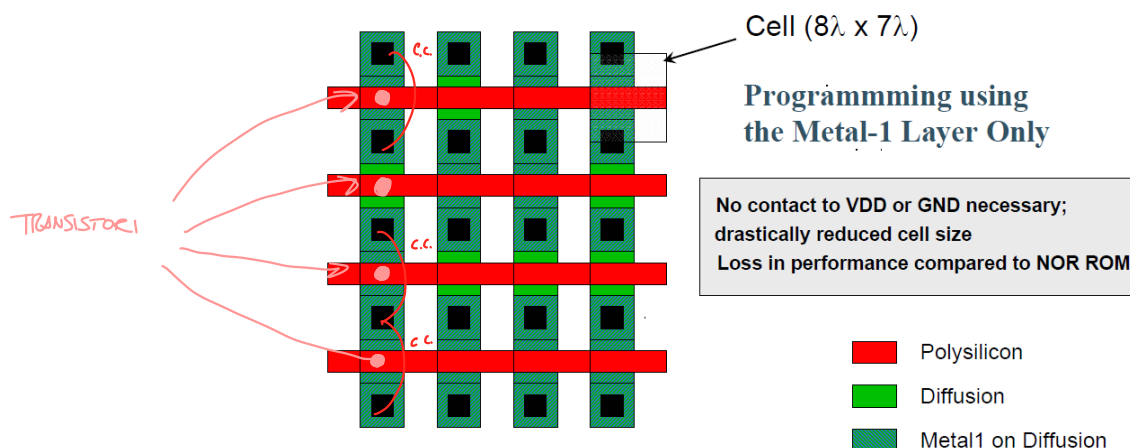
Data stored at
address 0,1,2,3 ?



All word lines high by default with exception of selected row

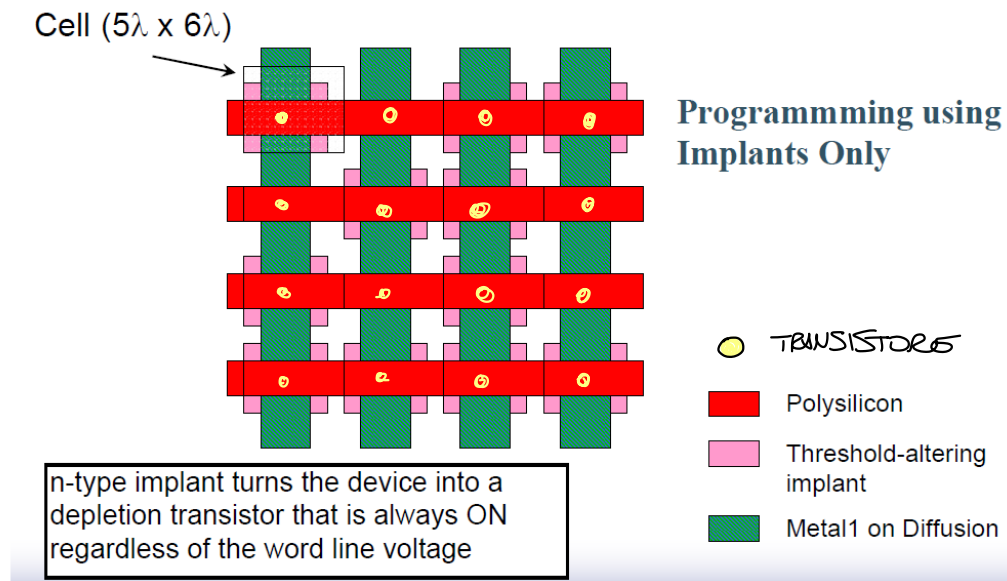
- il contatto c'è solo con i transistori di PU in quanto i transistori sono in serie
- più efficace dal punto di vista dell'area
- i mosfet che sono direttamente connessi, sono connessi da una connessione "naturale" in quanto sono comunque due tasche n+ in contatto
- architettura NAND → dove c'è il transistor viene impedita la scarica della BL. Tutte le WL sono alte tranne quella che voglio selezionare (nand)
- l'informazione consiste nel avere o non avere il transistor
- i transistori che non sono presenti in realtà sono dei transistori con soglia negativa che non possono essere mai spenti e quindi non possono essere selezionati

MOS NAND ROM Layout



- grande compattezza
- in rosso le WL vanno ai transistori (sono i gate)
- dove c'è la METAL1 on diffusion tra due transistori vuol dire che essi sono cortocircuitati (S=D) (non ci sono) pertanto per qualsiasi valore della WL essi sono sempre spenti
 - il corto circuito è nella terza direzione ovvero corre sopra
 - c'è bisogno di un contatto per ogni S e D → ce ne sono tanti (quadratinini neri)
- dove non c'è la metal vuol dire che c'è il transistor
- in questo layout i transistori che devono condurre sono quelli cortocircuitati

NAND ROM Layout



- non ci sono più contatti
- la presenza del transistore è ottenuta con l'alterazione della soglia
 - i transistori sono di tipo n con silicio di tipo p
 - viene impiantato del silicio di tipo n nel silicio di tipo p (colore rosa)
 - in pratica vengono connessi S e D (che sono n+)
 - è equivalente ad aver creato una tensione di soglia negativa (il transistor è sempre acceso)
- **l'architettura nand ha come svantaggio che le BL hanno connessioni verso massa attraverso dei transistori in serie: all'aumentare del numero di transistori tende a rallentare: il ritardo tende a crescere in maniera più che lineare (c'è una resistenza e una capacità)**

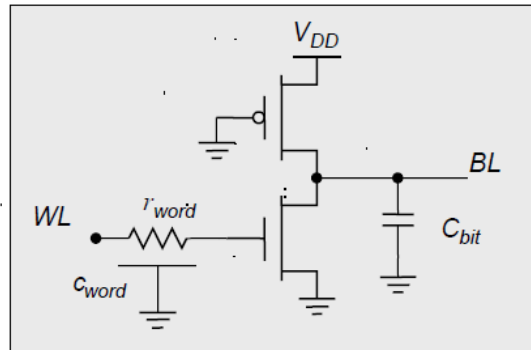
NOR → meno compatta (più contatti), più veloce (transistori in parallelo)

NAND → più compatta, più lenta

Modello equivalente MOS NOR ROM

Equivalent Transient Model for MOS NOR ROM

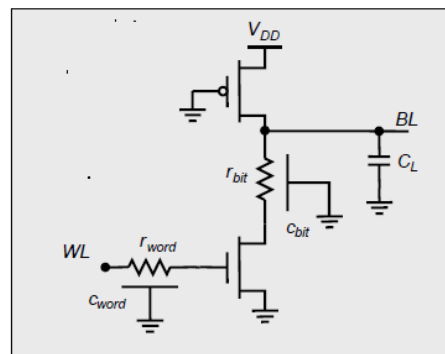
Model for NOR ROM



- Word line parasitics
 - Wire capacitance and gate capacitance
 - Wire resistance (polysilicon)
- Bit line parasitics
 - Resistance not dominant (metal)
 - Drain and Gate-Drain capacitance

Equivalent Transient Model for MOS NAND ROM

Model for NAND ROM



- Word line parasitics
 - Similar to NOR ROM
- Bit line parasitics
 - Resistance of cascaded transistors dominates
 - Drain/Source and complete gate capacitance

- la WL è na interconnessione: sistema distribuito di tipo RC che hanno una loro costante di tempo intrinseca: il ritardo aumenta più che linearmente con la lunghezza
- le WL sono realizzate il polisilicio e sono dello stesso materiale usato per il gate del transistor
- le BL sono metalliche → l'effetto di resistenza non è critico ma essendo lunga la capacità è grande alla quale si aggiunge la capacità parassita del MOS

NOR:

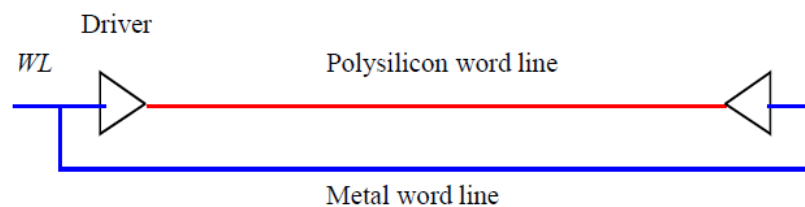
- il suo vantaggio è che se viene attivata la cella elementare (transistor n), questa produce una connessione tra BL e massa attraverso un solo transistor

NAND:

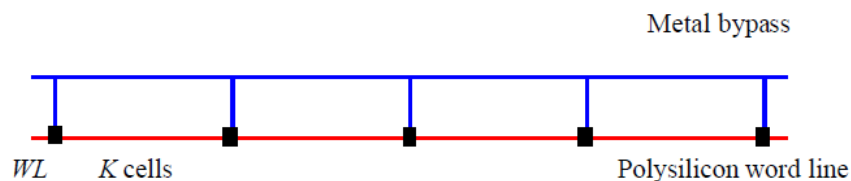
- per la BL si ha una connessione di MOS in serie che possono essere visti come un sistema distribuito RC che da un ritardo che cresce più che linearmente col numero di MOS.
- ogni mos è dato da una resistenza e una capacità
- se devo leggere grandi quantità di dati quello che conta è lo throughput: nelle memorie NAND pur avendo l'accesso alla singola informazione relativamente lento, hanno un parallelismo grande quindi lo throughput è grande ?

Decreasing WL delay

Decreasing Word Line Delay



(a) Driving the word line from both sides



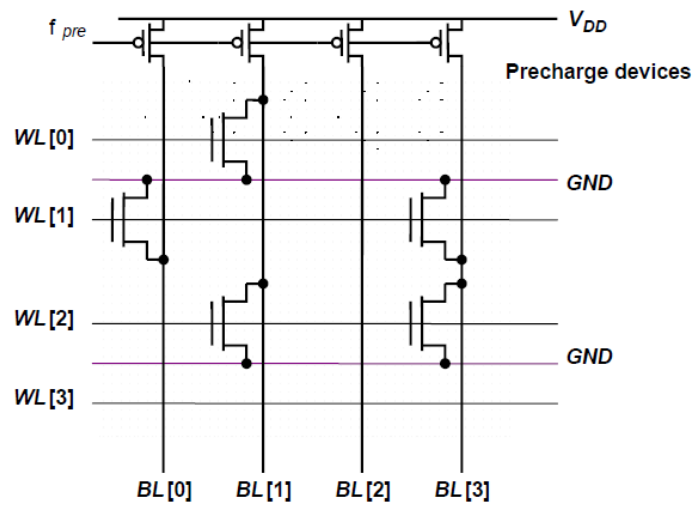
(b) Using a metal bypass

(c) Use silicides

- la distanza la misuro dal driver → se pilotato da due parti, inietta corrente dagli estremi
- metal bypass → sopra il transistor c'è un metallo che collega source e drain
 - l'aggiunta di metal bypass aiuta in quanto è come replicare gli ingressi alla WL e ho praticamente lo stesso segnale in tutta la WL
- siliciuri → sono delle leghe tra metalli e silicio (e quindi anche polisilicio)
 - la resistenza del source (n+) è una resistenza di accesso solitamente grande che dipende dalla profondità (verso il basso) della tasca
 - non può essere troppo profonda per lo scaling → il mod funziona male
 - i siliciuri cortocircuitano la tasca n+ → hanno conducibilità migliori del silicio

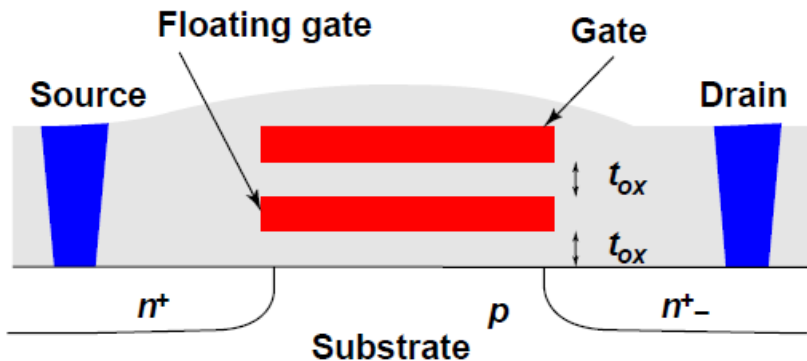
Precharged MOS NOR ROM

Precharged MOS NOR ROM

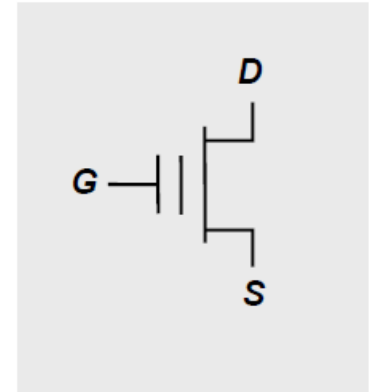


PMOS precharge device can be made as large as necessary, but clock driver becomes harder to design.

FLoating-gate transistor (FAMOS)

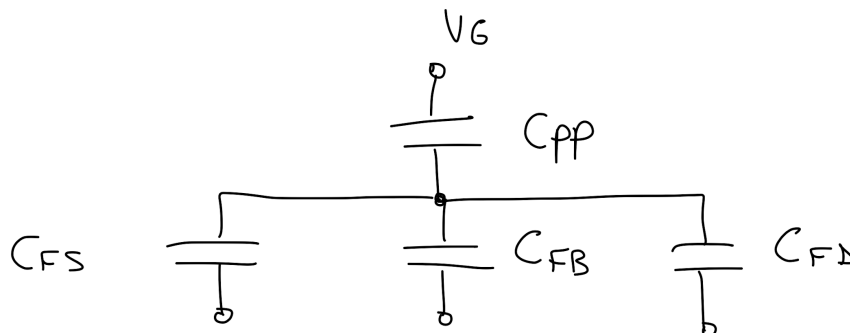


Device cross-section



Schematic symbol

- è un mosfet con due gate
 - **control gate** (quello sopra) → è quello accessibile che può essere polarizzato
 - **floating-gate** → pilota direttamente la zona attiva (polarizza il silicio, ovvero il mosfet) ma non ha una connessione perchè è circondato completamente dall'ossido (floating). Non è accessibile dall'esterno direttamente
 - è accoppiato capacitivamente sia col control gate (C_{pp} → parallel plate) che con le tasche del source e drain (C_{FD} , C_{FS} → floating drain e source di fringing) che con il silicio (C_{FB} → floating bulk)
- se prendiamo dei condensatori lineari, la carica nel floating gate è:
 - $Q_{FG} = C_{pp} (V_{FG} - V_G) + C_{FS} (V_{FG} - V_S) + C_{FD} (V_{FG} - V_D) + C_{FB} (V_{FG} - V_B)$
 - se $C_t = C_{pp} + C_{fs} + C_{fd} + C_{fb}$
 - $V_{fg} = (Q_{fg} / C_t) + C_{pp} \cdot V_G / C_t + C_{fd} \cdot V_D / C_t + C_{fs} \cdot V_S / C_t + C_{fb} \cdot V_B / C_t$
- lo stato on/off dipende direttamente dalla tensione al floating gate ma solo indirettamente da quella del control gate



$$V_{FG} = \frac{Q_{FG}}{C_T} + C_{pp} \cdot \frac{V_G}{C_T} + C_{FB} \frac{V_D}{C_T} + C_{FS} \frac{V_S}{C_T} + C_{FB} \cdot \frac{V_B}{C_T}$$

se $V_S = V_B = 0$

$$V_{FG} = \frac{Q_{FG}}{C_T} + C_{pp} \cdot \frac{V_G}{C_T} + C_{FB} \frac{V_D}{C_T}$$

$$V_T = V_{T,FG} \cdot \frac{C_T}{C_{PP}} - \frac{C_T}{C_{PP}} \cdot \frac{C_{FD}}{C_T} V_D - \frac{Q_{FG}}{C_{PP}}$$

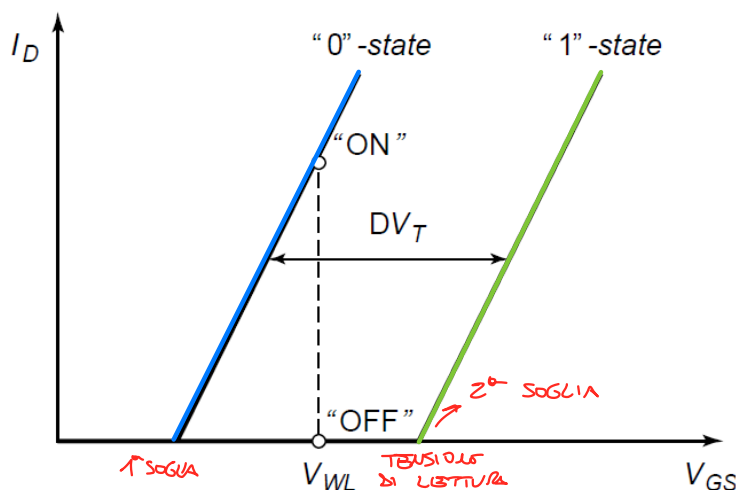
$$= \frac{V_{T,FG}}{\alpha_{FG}} - \frac{C_{FD}}{C_{PP}} \cdot V_D - \frac{Q_{FG}}{C_{PP}}$$

LETTURA: condizione in cui vado a testare la soglia

- supponiamo $V_s = V_b = 0$, $V_d = V_{dt}$
- quale tensione V_g sul gate devo applicare per accendere il transistor ovvero per far sì che $V_{fg} > V_t$?
- $V_{t,fg} = V_{t0}$
- V_t : V_g tale che $V_{fg} = V_{t,fg}$
- $V_t = V_{t,fg} \cdot C_t / C_{pp} - C_{fd} \cdot V_d / C_{pp} - Q_{fg} / C_{pp} = V_{t,fg} / \alpha_{fg} - C_{fd} \cdot V_d / C_{pp} - Q_{fg} / C_{pp}$
 - la V_t è sicuramente più grande di $V_{t,fg}$ in quanto c'è a denominatore il termine α_{fg} che è sicuramente < 1 (C_{pp} / C_t)
 - il drain contribuisce all'accensione del transistor, con l'accoppiamento capacitico il drain "spinge" il floating gate
 - se metto una carica negativa la soglia sale

↓ nell'equazione si riduce la tensione V_t da applicare nel gate

A "Programmable-Threshold" Transistor

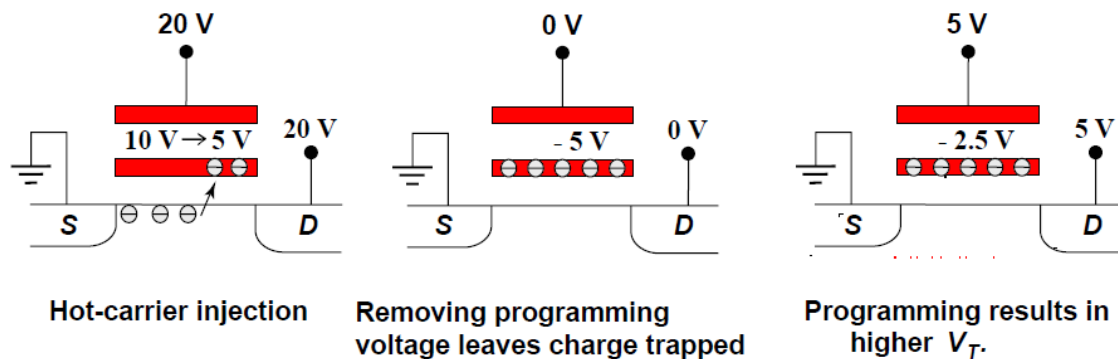


- se in questo dispositivo si vanno a iniettare elettroni e si riesce a modulare la carica nel floating gate, la modulazione della soglia è molto efficace. L'effetto è molto robusto
- se $Q = 0$ la tensione di lettura è maggiore della tensione di soglia e quindi il transistor si accende e leggo una corrente abbastanza alta
- se $Q > 0$ la tensione di lettura è minore della tensione di soglia (va programmata) e quindi essendo il transistor spento leggo una corrente pressochè nulla
- posso quindi associare allo stato '0' una corrente > 0 , e allo stato '1' una corrente nulla
- se la carica è sufficiente e gli ossidi sono abbastanza spessi, la carica rimane immagazzinata e non è volatile (ci vogliono 3 elettronvolt di barriera di potenziale)
 - il dato rimane valido per circa 10 anni

Programmazione floating-gate

Si tratta di iniettare una carica nel floating-gate

Floating-Gate Transistor Programming

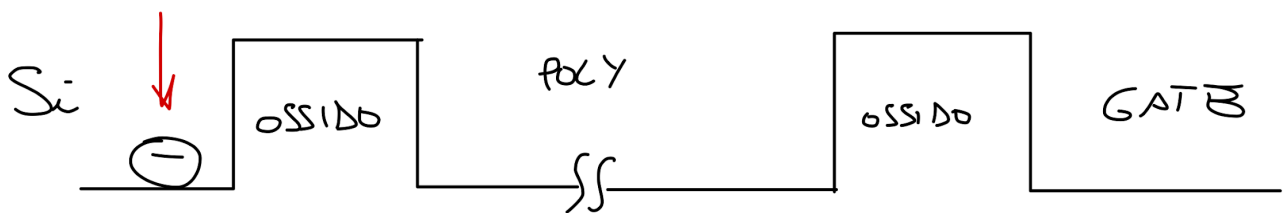


Come si riesce a far passare la carica attraverso l'ossido?

I portatori sono confinati nel semiconduttore (non passano attraverso l'ossido) in quanto l'ossido offre una barriera di energia potenziale molto alta affinché gli elettroni riescano a passare (circa 3 eV). Gli elettroni hanno energie acquisite dell'ordine di decine di meV.

Quando l'elettrone arriva all'ossido vede che l'energia che serve è molto alta e quindi torna indietro (turning point)

elettrone



1. **hot-carrier injection** → si trova un modo per cedere energia agli elettroni tale da poter passare sopra la barriera. Si applicano dei campi elettrici e delle tensioni tra source e drain che non sono consueti per il normale funzionamento dei mosfet (tensioni inconsuetamente grandi → 20V). Con una tensione V_g molto alta il transistor è molto acceso. In media il campo elettrico vale V_{ds}/L (lunghezza del transistor) che non è uniforme ma risulta "piccato" dalla parte del drain.
 - a. il campo elettrico è talmente intenso vicino al drain che gli elettroni saltano nel floating-gate rimanendo intrappolati (hanno ossido attorno)
 - b. occupazione di boltzmann $\exp(-E/kT)$ → si otterrebbe l'effetto anche scaldando

2. rimuovo queste tensioni dopo che gli elettroni sono stati inseriti nel floating-gate. Avendo messo carica negativa la tensione del FG va giù (nell'esempio 5V). La tensione di soglia è cresciuta molto

Funziona bene avendo un ossido spesso (qualche decina di nm)

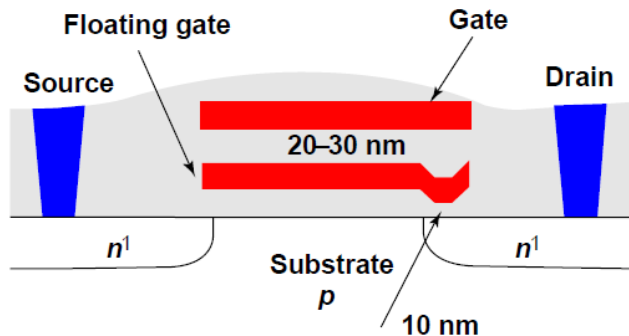
- buono per la **ritenzione degli elettroni**

Per tirare fuori gli elettroni

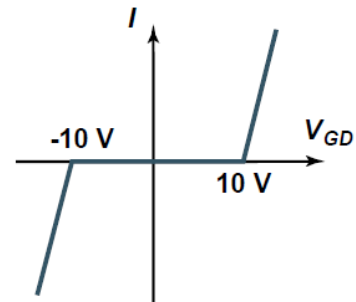
- il riscaldamento o campo elettrico non funziona (emula un metallo il floating gate)
- effetto tunnel se avevo ossidi poco spessi ($<10\text{nm}$)
- l'unico modo era toglierle dalla scheda e metterle sotto una lampada UV (optoelettronico)

Inoltre l'iniezione di elettroni tende a rallentare l'iniezione stessa in quanto il processo è autolimitante: quando inietto elettroni nel floating gate, la tensione nel floating gate si abbassa e questo tende a spegnere l'iniezione (tende a spegnere il transistor e ricade il campo elettrico che è favorevole ad attrarre elettroni nel floating gate). Se non fosse limitante porterebbe ad eventi fisici che danneggerebbero la cella stessa

FLOTOX EEPROM



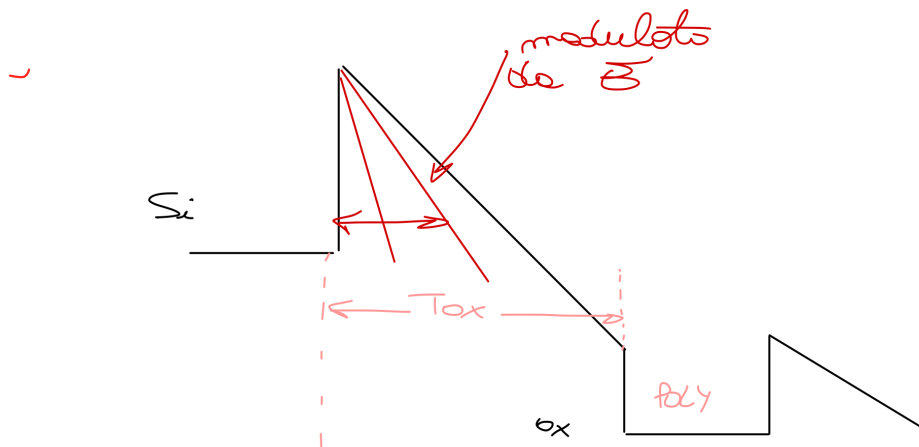
FLOTOX transistor



**Fowler-Nordheim
I-V characteristic**

- FLOating Tunneling OXide
- nella regione in cui il floating gate si sovrapponeva con una regione n+ (ad esempio col drain) c'è una regione con un ossido più sottile.
- il meccanismo di attraversamento ora è di tipo **tunneling** (campo elettrico)
 - è un meccanismo bidirezionale ovvero posso estrarre o introdurre elettroni semplicemente invertendo il campo elettrico
 - la probabilità di attraversare la barriera di potenziale decresce esponenzialmente con l'altezza della barriera e lo spessore della barriera stessa
 - se l'ossido però è troppo sottile ho rischi di leakage
 - fissato lo spessore fisico dell'ossido, posso modulare la corrente di elettroni con il campo elettrico in quanto riduco il cammino degli elettroni (distanza di tunneling → distanza che determina la probabilità che un elettrone possa attraversare la barriera)
 - rendere la barriera "trasparente"

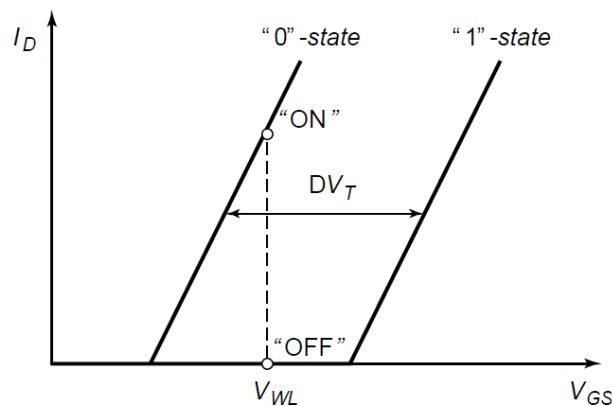
Grafico → se applico una tensione sufficientemente alta allora passerà corrente (di elettroni da silicio a floating gate). Funziona anche dalla parte opposta e quindi il dispositivo diventa electrically erasable EEPROM



Se applico una tensione negativa, praticamente Si e floating gate (nello schema “poly”) si invertono e quindi sta volta gli elettroni modulando il campo E vanno dal FG al silicio. Per la lettura leggo la corrente, con una tensione della WL a seconda di dove è la soglia leggo la corrente o non la leggo

Se programmo miliardi di celle non si può pensare che abbiano tutte la stessa soglia; questo diventa un problema in quanto, guardando al grafico di lettura, quelle con soglia bassa formeranno un “fascio di corrente” così come quelle con soglia alta e le cose andranno bene fintanto che le soglie del livello alto e del livello basso non iniziano a somigliarsi. La distribuzione delle soglie è una figura di merito molto importante.

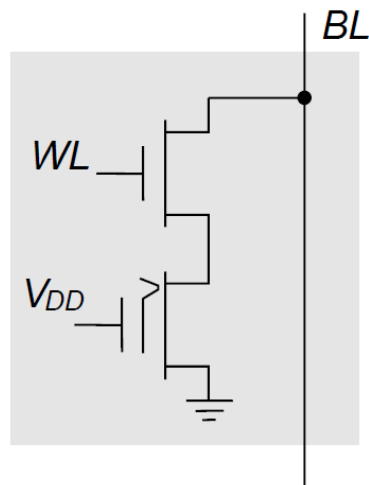
A “Programmable-Threshold” Transistor



Nelle FLOTOX (Fowler-Nordheim) il controllo della tensione di soglia è poco preciso. C'è una dispersione statistica molto grande e spesso certe celle si trovano ad avere una soglia negativa e non è più possibile spegnerle (**overerasing** → **le celle sono state cancellate troppo**)

- le WL non indirizzate sono a zero e le celle sulle WL non indirizzate conducono se la soglia è negativa quindi è un effetto indesiderato

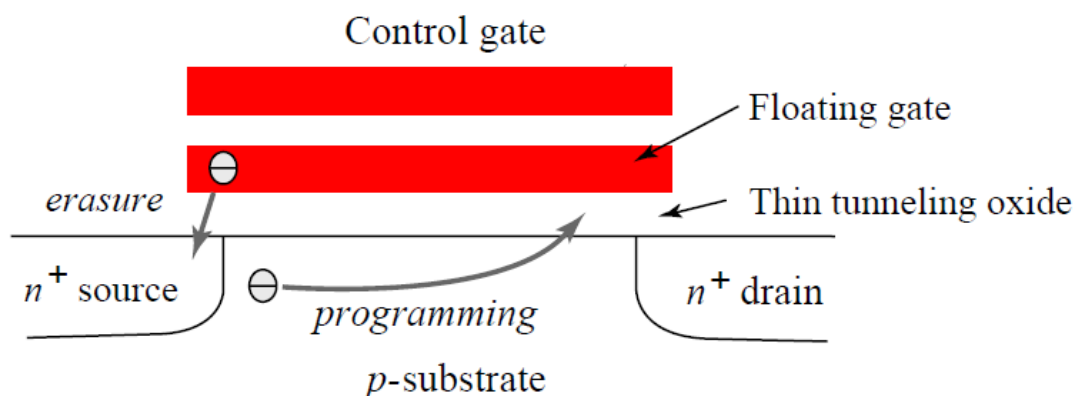
EEPROM CELL



Absolute threshold control
is hard
Unprogrammed transistor
might be depletion
⇒ **2 transistor cell**

- si mette in serie al floating gate (quello col beccuccio e i due gate) un **access transistor**
- quando si ha una cella della quale si fa fatica ad assicurare che sia spenta quanto viene applicata una tensione pari a zero, l'access transistor (ha la sua soglia) si prende la WL
- le celle EEPROM sono più grandi delle EPROM

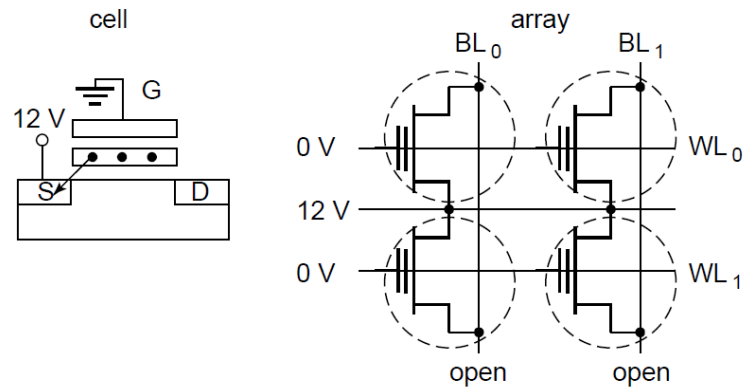
Flash EEPROM



- unisce le caratteristiche delle FAMOS e delle FLOTOX (elettroni caldi e tunneling)
- hanno un ossido sottile tra floating gate e substrato (circa 10nm)
- la programmazione avviene per elettroni caldi (source a massa, gate e drain a tensioni elevate)
- la cancellazione avviene con un meccanismo di tunneling confinato al source: questo è legato alle tensioni che vengono usate (control gate a massa e sulla source line si tira su la tensione a >10V)
 - gli elettroni vengono attratti dal source che ha tensione alta e positiva
 - il drain non è polarizzato → annulla le correnti e consente di realizzare la cancellazione sull'intera matrice
 - a livello di blocco viene cancellato l'intero blocco (la source line è unica)
- c'è anche qua il problema delle soglie (OVERERASING)

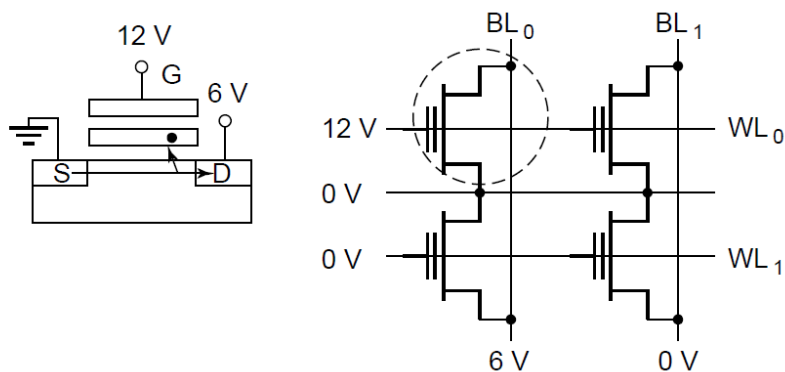
NOR

Basic Operations in a NOR Flash Memory— Erase



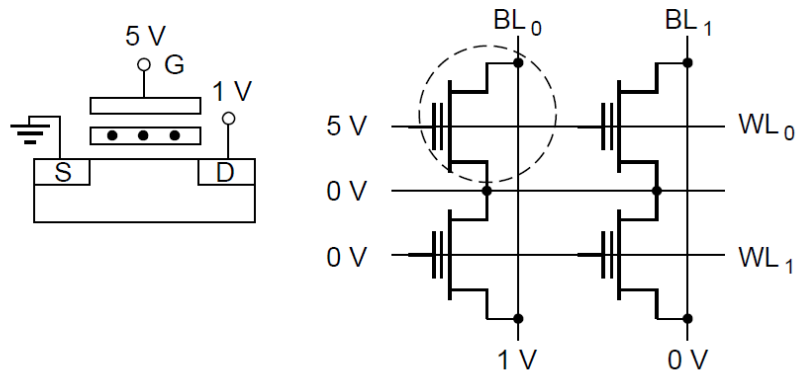
- la linea di 12V è la **source line**
- non c'è il transistor di selezione
- i gate sono a massa, i drain non sono contattati e il source è a 12V
- se si dà a tutte le celle uno stesso impulso di cancellazione, quelle che erano scritte e quelle che non erano scritte avranno una soglia diversa
 - un modo è quello di cercare di uniformare lo stato delle celle prima di cancellarle
 - si tende a portare positive le soglie prima di cancellare

Basic Operations in a NOR Flash Memory— Write



- drain e gate alti, source a massa

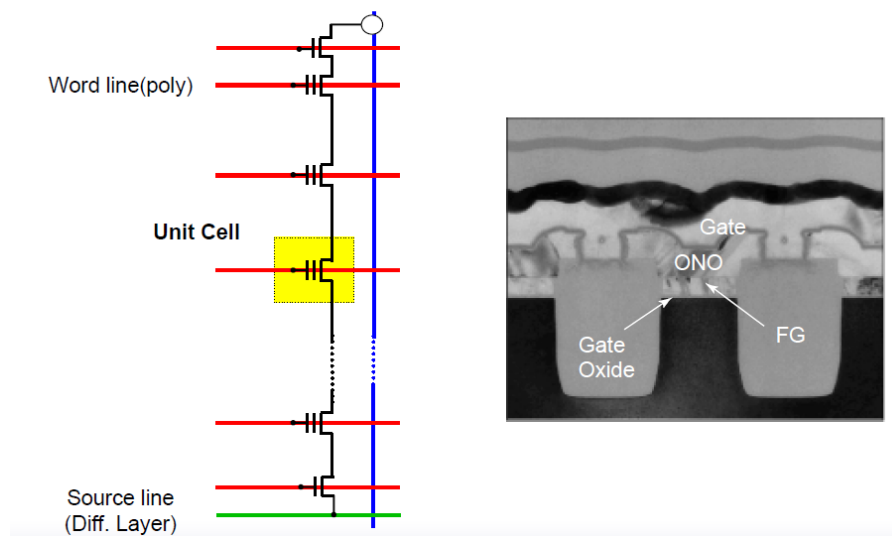
Basic Operations in a NOR Flash Memory— Read



- se i mosfet hanno tensione di soglia negativa conducono (non va bene)

NAND

NAND Flash Memory



- sono le memorie con cui si arrivano a fare gli SSD
- la cella di memoria è impilata (espansione in verticale)
- per leggere il contenuto di una cella, devono essere accese tutte le altre
 - tutte le celle non indirizzate devono avere WL alta e soglia più bassa della WL
 - la cella che voglio leggere ha una WL più bassa
- gate a zero, source alto per la cancellazione

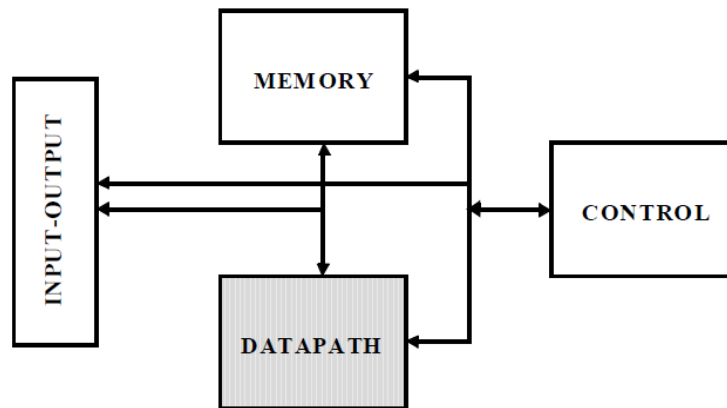
Programmazione multilivello: se in un dispositivo fisico riesco ad immagazzinare più di due livelli (vedi il grafico di lettura, e immagina di avere 4 linee diagonali invece che due → avrei 4 valori quindi 2 bit)

Table 12-1 Comparison between nonvolatile memories ([Itoh01]).

$V_{DD} = 3.3$ or 5 V; $V_{PP} = 12$ or 12.5 V.

	Cell— Nr. of Transistors	Cell Area (ratio wrt EPROM)	Mechanism		External Power Supply		Program/ Erase Cycles
			Erase	Write	Write	Read	
MASK ROM	1 T (NAND)	0.35–5	—	—	—	V_{DD}	0
EPROM	1 T	1	UV Exposure	Hot electrons	V_{PP}	V_{DD}	~100
EEPROM	2 T	3–5	FN Tunneling	FN Tunneling	V_{PP} (int)	V_{DD}	10^4 – 10^5
Flash Memory	1 T	1–2	FN Tunneling	Hot electrons	V_{PP}	V_{DD}	10^4 – 10^5
			FN Tunneling	FN Tunneling	V_{PP} (int)	V_{DD}	10^4 – 10^5

Digital Adders



- nella control processing unit vengono effettuate operazioni di tipo logico e aritmetico (datapath + control nello schema)
- questa architettura è quella di von neumann
- nella ALU (dentro il datapath) ci sono gli adders

Building Blocks for Digital Architectures

Arithmetic unit

- Bit-sliced datapath (adder, multiplier, shifter, comparator, etc.)

Memory

- RAM, ROM, Buffers, Shift registers

Control

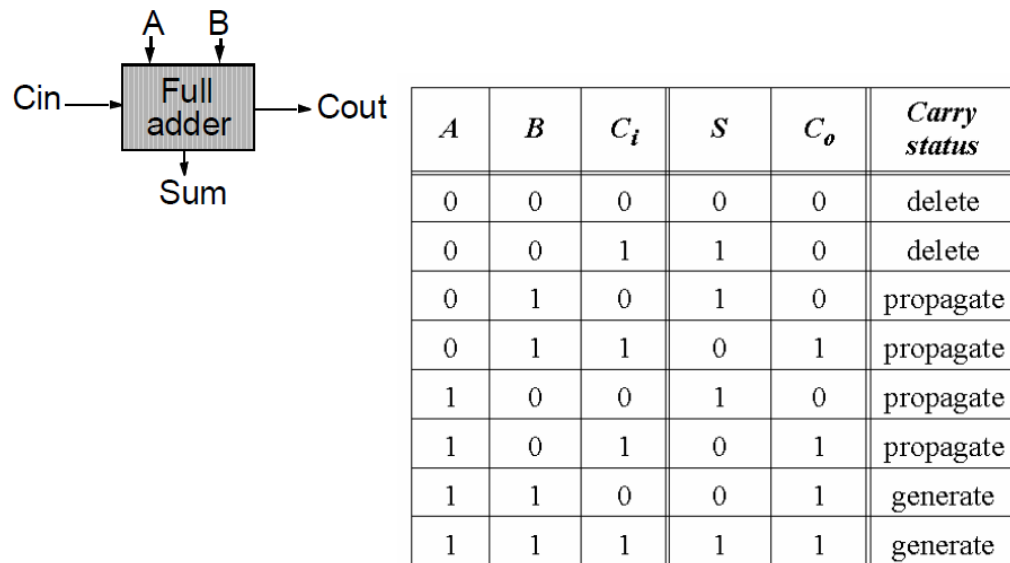
- Finite state machine (PLA, random logic.)
- Counters

Interconnect

- Switches
- Arbiters
- Bus

•

Full Adder



- è il blocco elementare per la somma di numeri di n bit (somma due bit)

$$S = A \oplus B \oplus C_i$$

$$= \bar{A}\bar{B}C_i + \bar{A}B\bar{C}_i + A\bar{B}\bar{C}_i + ABC_i$$

$$C_o = AB + BC_i + AC_i$$

•

Define 3 new variable which ONLY depend on A, B

Generate (G) = AB

Propagate (P) = A $\oplus$ B

Delete = $\bar{A}\bar{B}$

$$C_o(G, P) = G + PC_i$$

$$S(G, P) = P \oplus C_i$$

Can also derive expressions for S and C_o based on D and P

Note that we will be sometimes using an alternate definition for

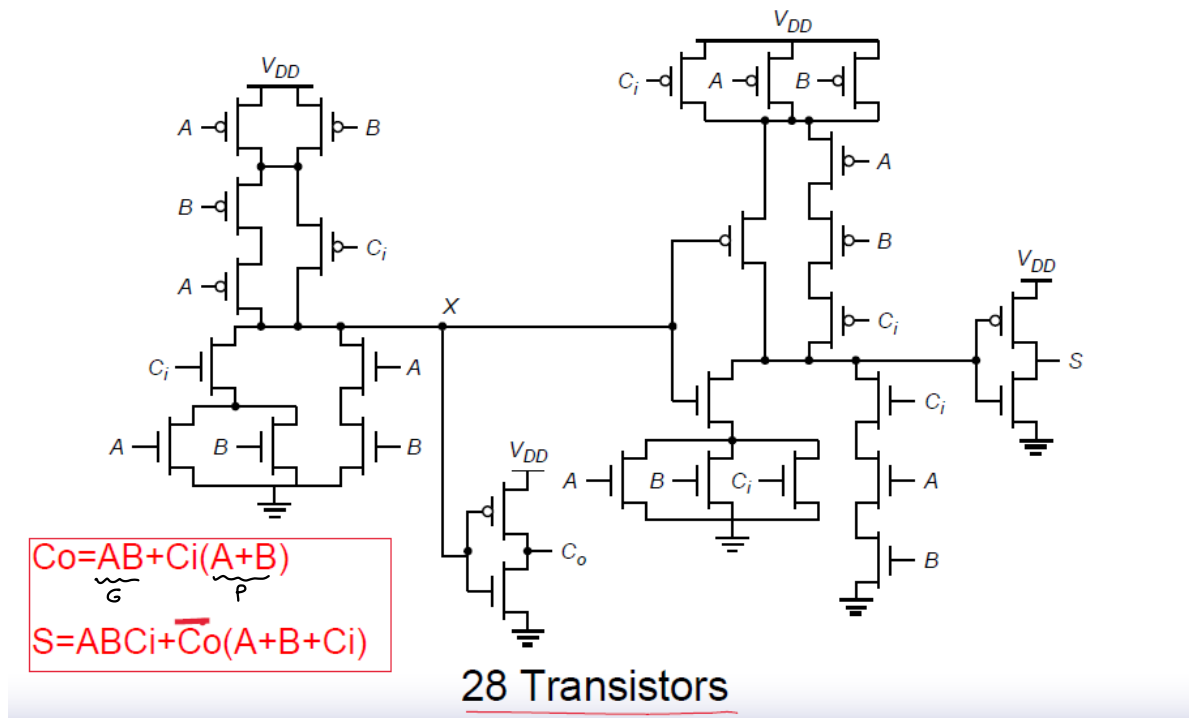
Propagate (P) = A + B

•

- **delete** → assicura che il carry out è zero indipendentemente dal carry in (se fosse uno lo cancella) (not A and not B)
- **propagate** → propaga il riporto in uscita (A xor B)
- **generate** → si genera il riporto indipendentemente dal riporto in ingresso (A and B)

CMOS Full Adder

Complimentary Static CMOS Full Adder



- $x = \text{not } C_o$
- utilizzo il calcolo di C_o per il calcolo della somma in modo da ridurre il numero di transistor (infatti non servirebbe)
- il calcolo di C_o nel circuito di sinistra è uguale a quello definito prima (guarda il PD e vedi che fa $(AB + C_i(A+B))$)

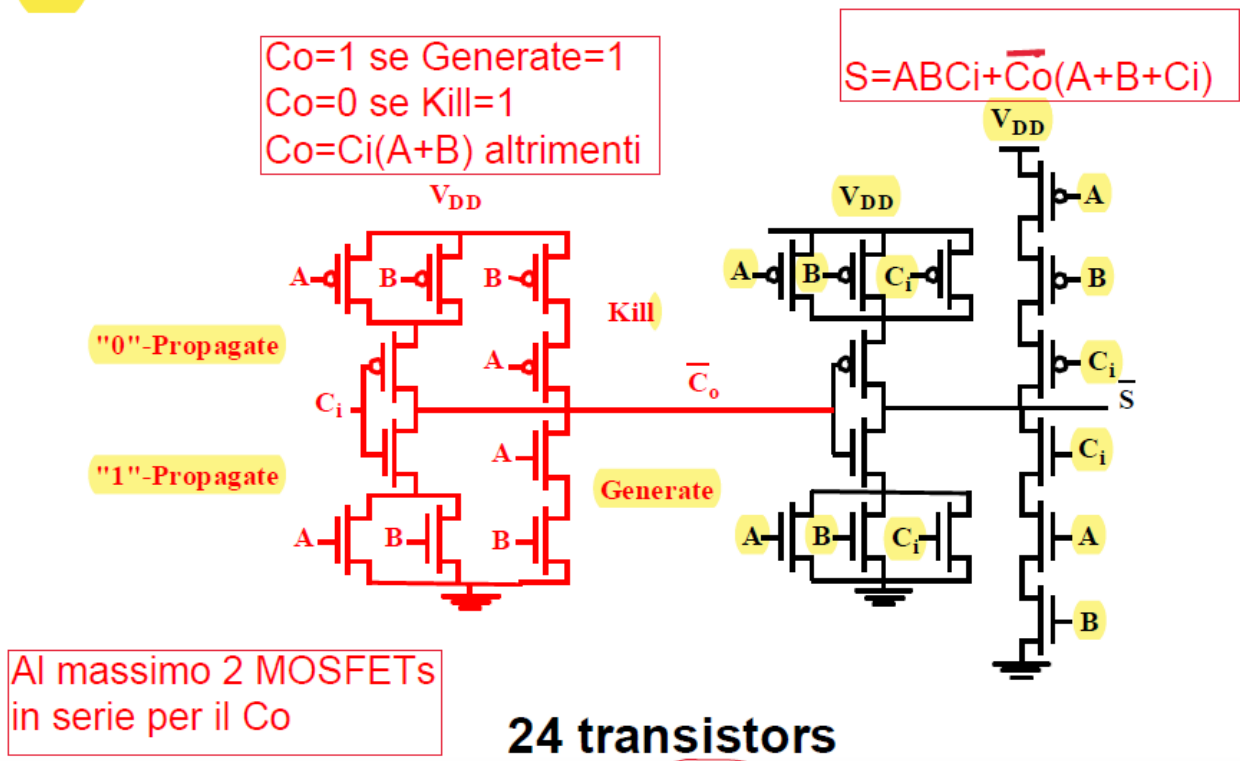
$$S = ABC_i + (!A!BC_i + !AB!C_i + A!B!C_i)$$

$$C_o = AB + AC_i + BC_i \rightarrow !C_o = !(AB) \text{ and } !(AC_i) \text{ and } !(BC_i) = (!A + !B) (!B + !C_i) (!A + !C_i) = !A!B + !A!B!C_i + !A!C_i + !B!C_i$$

$$!C_o(A+B+C_i) = !A!B!C_i + BC_i!A + !A!BC_i \text{ che è uguale alla seconda parte dell' "S" che sarà quindi } S = ABC_i + !A!B!C_i + BC_i!A + !A!BC_i$$

- nel circuito di destra guardando al PD ho proprio questa funzione (ricorda che il PD dà la funzione negata infatti dopo c'è un invertitore)
- se si dovesse implementare la funzione logica originale senza il C_o nel S (che è una somma di 4 termini ognuno composto da 3 ingressi) avrei $(3 \cdot 4) \cdot 2 + 10$ transistor per il calcolo di C_o + gli invertitori
- il bit di somma dovrà aspettare il C_o che è quello che si deve propagare e che dà il percorso critico quindi anche se la somma aspetta C_o non è un problema
- in questo schema ci sono delle criticità come ad esempio le lunghe serie di transistori in serie

A Better Structure: The Mirror Adder



- il PU e il PD sono speculari
- l'organizzazione speculare di un gate cmos non è così banale ovvia, la condizione necessaria per una sintesi corretta di un gate cmos è quella che la funzione logica di PU e PD siano l'una il negato dell'altra. Questo assicura che PU e PD siano attivi in modo mutuamente esclusivo (ci sarebbe una contesa nel pilotaggio dell'uscita)
- questi gate non sono duali ma soddisfano la condizione dei CMOS ovvero che PU e PD siano attivi in modo mutuamente esclusivo
- di buono c'è il fatto che per il bit di C_o adesso il circuito che lo produce ha al massimo due transistori in serie sia per salita che per discesa
- kill = delete
- il circuito a sinistra è composto da tre "pezzi" che gestiscono generate, delete e propagate (guarda immagine) per la generazione di $!C_o$
 - $C_o = G + C_i(A+B)$
- il circuito a destra produce $!S$
 - $!S = ABC_i + C_o(A+B+Ci)$
- E' diminuito il numero di transistori ma soprattutto si è snellita la generazione del C_o (per il fatto che ho al caso peggiore 2 transistori in serie nel PU e nel PD)
- il C_i è collegato molto vicino all'uscita (è sempre il transistor più vicino al nodo di uscita); infatti se per esempio nel PD del nodo S, A e B sono a uno e dovrebbe esserlo anche C_i ma esso non è ancora arrivato, nel frattempo il source del transistor di C_i sarà già stato scaricato a massa. Quando arriva C_i c'è solo un transistor da scaricare. Se fosse stato il più lontano, le capacità intermedie non avrebbero potuto scaricarsi a massa. **E' sempre meglio avere i segnali critici vicino al nodo di uscita**

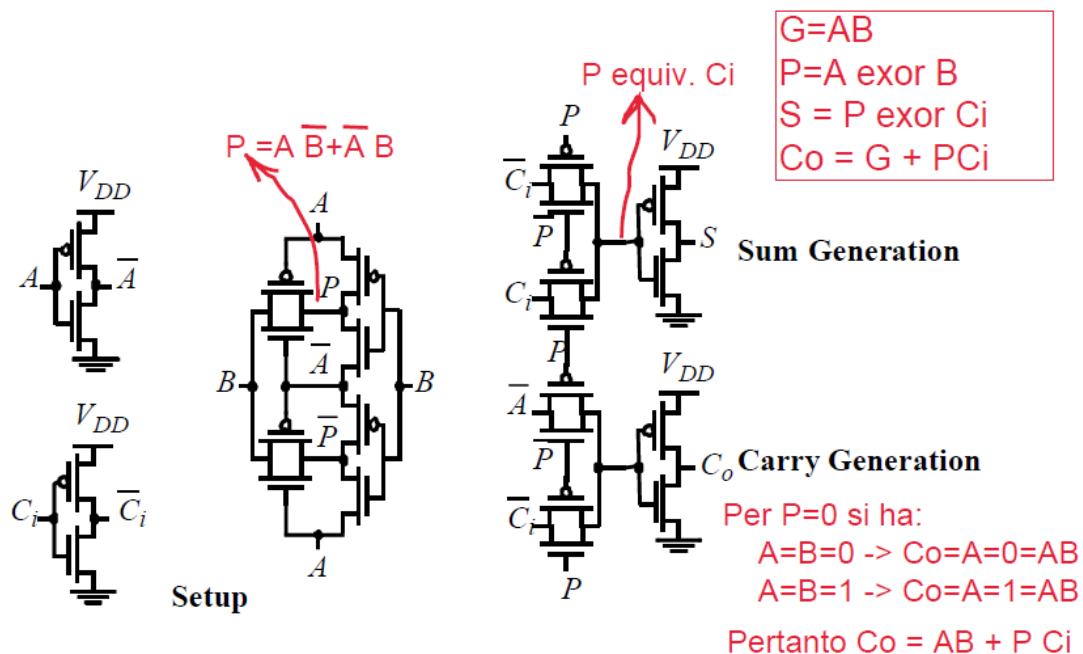
The Mirror Adder

- The NMOS and PMOS chains are **completely symmetrical**. A maximum of two series transistors can be observed in the carry-generation circuitry.
- When laying out the cell, the most critical issue is the minimization of the capacitance at node C_o . The reduction of the diffusion capacitances is particularly important.
- The capacitance at node C_o is composed of four diffusion capacitances, two internal gate capacitances, and six gate capacitances in the connecting adder cell .
- The transistors connected to C_i are placed closest to the output.
- Only the transistors in the carry stage have to be optimized for optimal speed. All transistors in the sum stage can be minimal size.

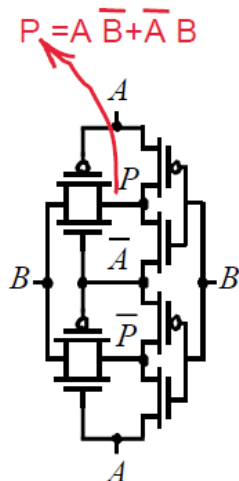
•

- il nodo C_o è elettricamente pesante nel senso che ha collegate 4 capacità di diffusione delle giunzioni (a sinistra) due capacità interne e sei capacità del gate che segue

Transmission gate Full Adder

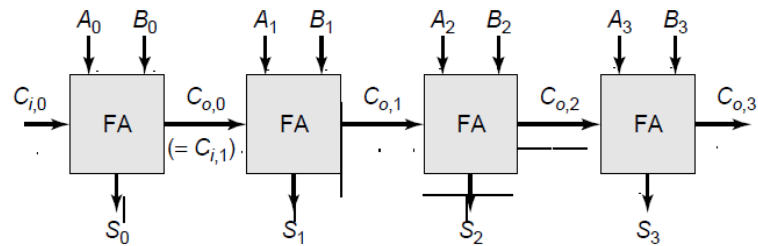


- essendo incentrato sullo XOR è bene utilizzare i pass-transistor di tipoi transmission gate (lo XOR può essere visto come un multiplexer portando in uscita b se a è negato ad esempio)
- ci sono sia P che !P in quanto servono entrambi negli step successivi
- in questo schema si fa uso del propagate ($A \text{ xor } B$)
 - se $A=0$ è acceso il pass-transistor in alto e viene portato B in P (l'invertitore a destra di P è in alta impedenza in quanto le alimentazioni sono girate)



- se $A=1$ il pass-transistor è spento e viene portato !B in P dall'invertitore
- il negato dell'xor è l'**equivalence** ($AB + !A!B$)
- Sum
 - $S = P \text{ exor } C_i$
 - il nodo intermedio implementa l'equivalence, infatti il pass-transistor sopra è di tipo !P che fa passare !C_i, quello sotto è di tipo P che fa passare C_i
- carry out
 - $C_o = G + P C_i$
 - Se $P = 0$ A e B devono essere uguali in quanto $P = A \text{ exor } B$
 - $A=B=0 \rightarrow C_o = A = 0 = AB$
 - $A=B=1 \rightarrow C_o = A = 1 = AB$
- il punto di forza sta nel numero ridotto di transistor, pochi transistori in serie

Ripple-Carry Adder



Worst case delay linear with the number of bits

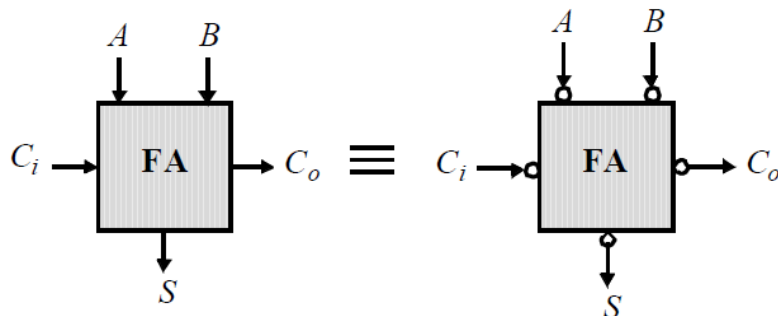
$$t_d = O(N)$$

$$t_{adder} = (N-1)t_{carry} + t_{sum}$$

Goal: Make the fastest possible carry path circuit

- la catena dei riporto è il percorso critico
- t_{sum} → tempo necessario per il singolo FA per generare la somma
- t_{carry} → tempo necessario per il singolo FA per generare il carry in uscita
- il carry si deve propagare per $N - 1$ FA

Inversion Property



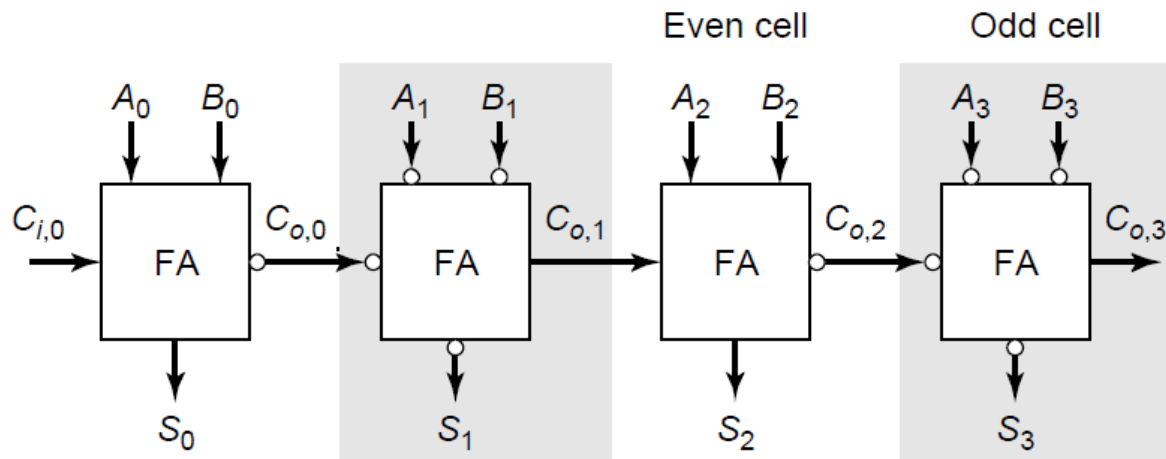
$$\bar{S}(A, B, C_i) = S(\bar{A}, \bar{B}, \bar{C}_i)$$

$$\bar{C}_o(A, B, C_i) = C_o(\bar{A}, \bar{B}, \bar{C}_i)$$

- il negato della somma degli ingressi in forma vera è la somma degli ingressi in forma negata
- stessa cosa per il carry out
- vale solo in questo caso

Se questa cosa è vera allora posso “mangiarmi l’invertitore in uscita

Minimize Critical Path by Reducing Inverting Stages

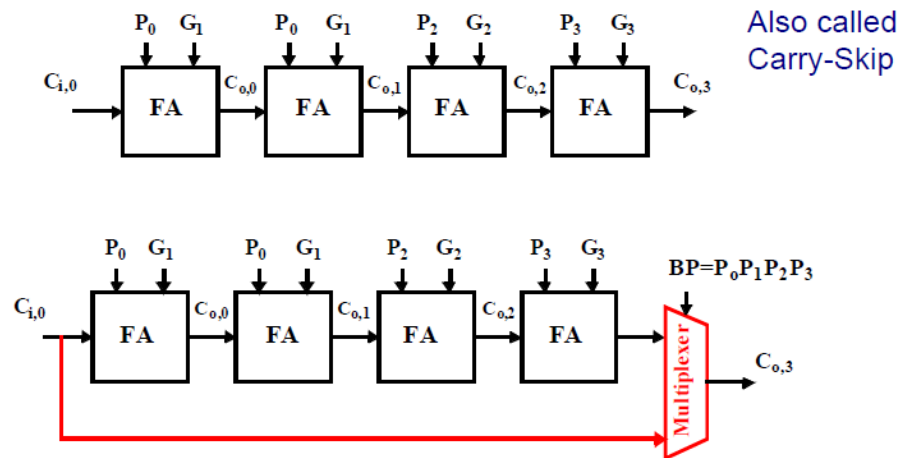


Exploit Inversion Property

- dal punto di vista del ritardo di propagazione viene migliorato in quanto gli inverter che servono per negare gli ingressi, rendono disponibile l'ingresso agli stadi quando il primo stadio sta propagando il carry out del primo stadio
- primo stadio → carry negato
- secondo stadio → ingressi negati → carry dritto e somma negata

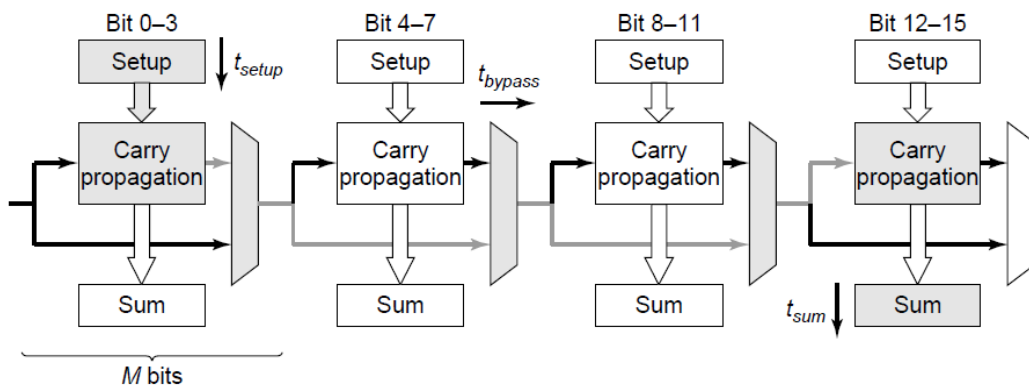
eur

Carry Bypass Adder



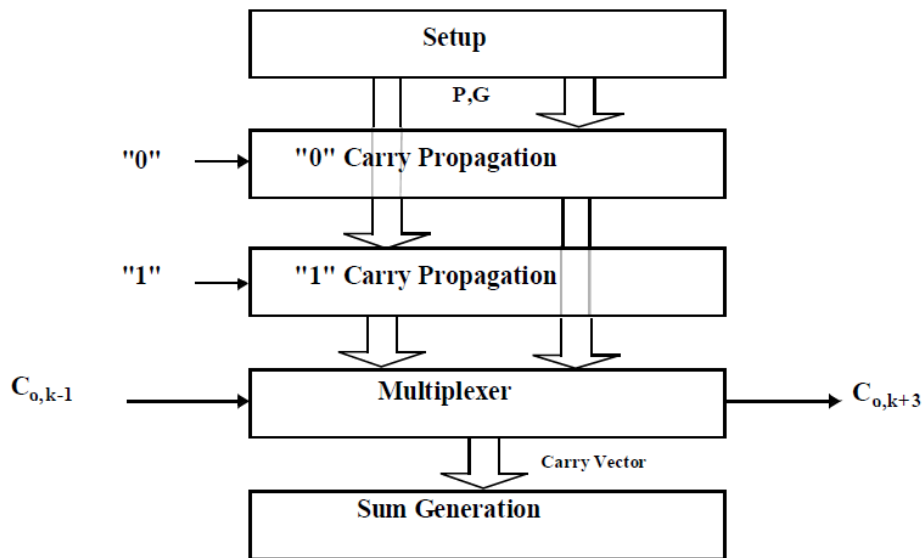
Idea: If (P_0 and P_1 and P_2 and $P_3 = 1$)
then $C_{o3} = C_0$, else “kill” or “generate”.

Carry-Bypass Adder (cont.)



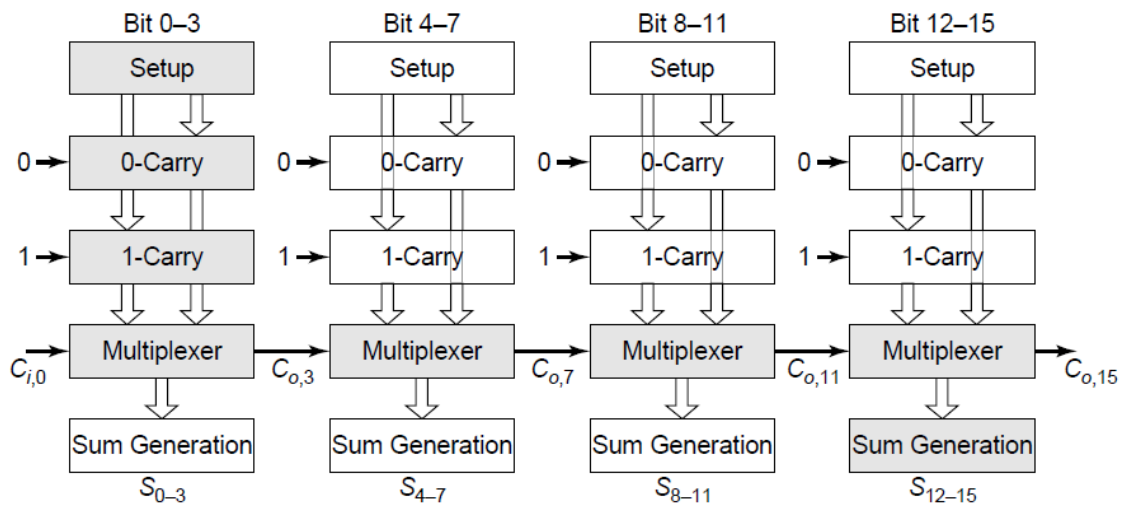
$$t_{adder} = t_{setup} + \underbrace{M t_{carry}}_{\text{TOTAL}} + \underbrace{(N/M - 1) t_{bypass}}_{\text{STABLE}} + \underbrace{(M - 1) t_{carry}}_{\text{ULTIMO STABLE}} + t_{sum}$$

Linear Carry Select Adder



- nell'ottica di guadagnare tempo, calcolo tutte e due le possibilità di carry out (o zero o uno)
- con un multiplexer seleziono quello giusto quando mi arriva il carry in

Carry Select Adder: Critical Path

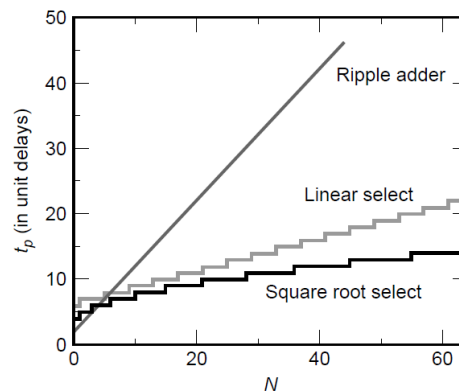


- divido il numero di 16 bit in 4 gruppi → gli ingressi arrivano ai 4 stadi contemporaneamente e possono andare avanti in parallelo (setup degli ingressi)
- tutti gli stadi calcolano il carry out sia nel caso che il Cin sia 0 che nel caso sia 1 (cin non è ancora arrivato)
- quando arriva il vero carry in nel primo stadio, calcola la somma e manda il carry out allo stadio successivo che ha già calcolato tutto e deve solo fare il multiplexing a seconda del carry che gli arriva dal primo stadio

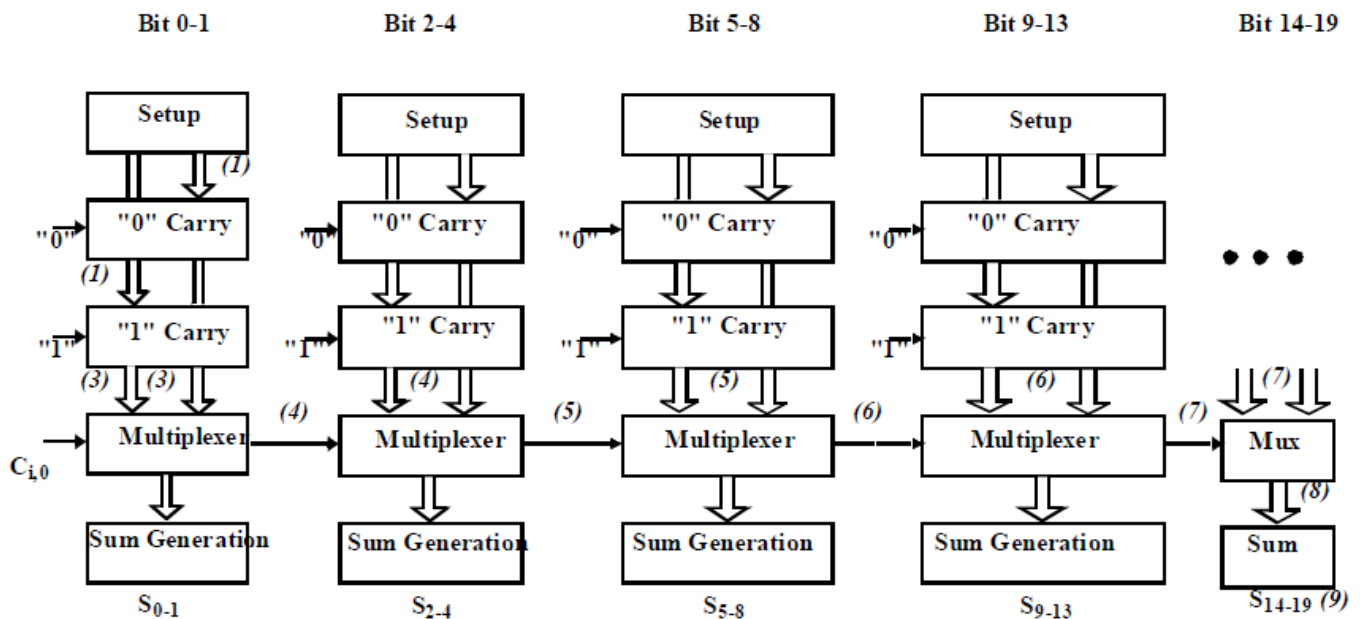
$$t_{add} = t_{setup} + M t_{carry} + \frac{N}{M} t_{mux} + t_{sum}$$

- N = numero di bit, M = numero di bit in ogni sottogruppo, t_{setup} = tempo necessario per il setup dei bit di ingresso, t_{carry} = ritardo del primo gruppo per produrre il carry (sono M bit), t_{mux} = ritardo legato ai multiplexer (che sono N/M)
- quello che guardiamo è la dipendenza da N numero di bit; se t_{mux}/M è apprezzabilmente più piccolo di t_{carry} (che era quello legato a N nel ripple adder) allora il ritardo di propagazione crescerà più lentamente all'aumentare del numero di bit

Adder Delays - Comparison



Square Root Carry Select



$$t_{\text{add}} = t_{\text{setup}} + P \cdot t_{\text{carry}} + (\sqrt{2N}) t_{\text{mux}} + t_{\text{sum}}$$

- il più a sinistra ha soltanto due bit, quello immediatamente successivo ne prende 3, e così via
- è inutile che i blocchi più a destra finiscano assieme al primo perchè tanto poi rimangono in attesa
- rendo il blocco di sinistra meno profondo e quindi più veloce, di fatto la velocità di propagazione diminuisce gradualmente verso destra

- ogni blocco dovrebbe essere dimensionato in modo che l'ultimo finisca il setup e la generazione dei due carry out poco prima che arrivi il carry in
- la dipendenza del ritardo da N non è più lineare (radice quadrata di N)